

UNIVERSIDAD DE SANTIAGO DE
COMPOSTELA



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA

Estudio de mecanismos cuánticos para la optimización de hiperparámetros en redes neuronales profundas

Autor/a:

Diego Suárez Castro

Tutores:

Manuel Lama Penin

Anselmo Tomás Fernández Pena

Grado en Inteligencia Artificial

Julio 2026

Trabajo de Fin de Grado presentado en la Escuela Técnica Superior de Ingeniería de la Universidad de Santiago de Compostela para la obtención del Grado en Inteligencia Artificial.

Agradecimientos

A mis tutores Manuel y Tomás por haberse prestado a esta colaboración, por su apoyo y disponibilidad durante el proyecto.

A mi familia, pareja y amigos por hacer el camino más ameno estos años.

Resumen

Este trabajo investiga el potencial de la computación cuántica para la optimización de hiperparámetros (HPO) en modelos de aprendizaje profundo, centrándose específicamente en arquitecturas de tipo Transformer. El objetivo principal es explorar la viabilidad de algoritmos cuánticos variacionales bajo las restricciones de la actual era NISQ y compararlos frente a baselines clásicos consolidados, como la Optimización Bayesiana, y baselines cuánticos como el VQE estándar.

Mediante el uso intensivo de los recursos de supercomputación del CESGA (FinisTerra III) y la posterior validación en el ordenador cuántico real Qmio, se ha ejecutado un estudio empírico de escalabilidad que abarca desde los 5 hasta los 32 qubits. Esta transición desde la simulación numérica hacia la ejecución en hardware físico real permite evaluar el comportamiento de los algoritmos cuánticos, bajo condiciones genuinas de ruido y decoherencia. Los resultados experimentales demuestran que el algoritmo híbrido QITE-VQE (Quantum Imaginary Time Evolution) no solo muestra una resiliencia excepcional ante modelos realistas de ruido térmico y despolarización, sino que logra esquivar los denominados *Barren Plateaus* (mínimos locales catastróficos) que penalizaron severamente al VQE estándar y a la búsqueda clásica en determinadas configuraciones del espacio de estados.

El estudio concluye que QITE-VQE opera en la frontera de eficiencia de Pareto óptima, logrando una convergencia superior en una fracción del coste temporal asintótico que requeriría la evaluación clásica de espacios masivos (2^{32} combinaciones). En definitiva, este trabajo valida empíricamente la eficacia de los filtros de tiempo imaginario para estabilizar la optimización cuántica en procesadores reales, posicionando esta tecnología como una herramienta estratégica y escalable para el futuro diseño de la Inteligencia Artificial de alta escala.

Memoria tipo A – Índice general

1. Introducción	1
1.1. Computación cuántica	2
1.2. Algoritmos cuánticos	4
1.3. Objetivos del trabajo	5
1.4. Implicaciones Transversales del Estudio	6
1.5. Organización de la memoria	6
2. Estado de conocimiento del problema a abordar	9
2.1. Estado del arte de HPO	9
2.1.1. Hiperparámetros relevantes	9
2.1.2. Algoritmos Clásicos de Optimización y Fine-Tuning	10
2.2. Estado del arte en computación cuántica	11
2.2.1. QITE-VQE	13
2.2.2. Posicionamiento del Trabajo	13
3. Materiales	15
3.1. Recursos Hardware	15
3.1.1. Clúster de Supercomputación FinisTerra III (CESGA)	15
3.1.2. Ordenador Cuántico Qmio	16
3.1.3. Hardware de Desarrollo Local	17
3.2. Conjuntos de datos	17
3.2.1. Oráculo de Rendimiento y Paisaje de Optimización Subro- gado	17
3.2.2. Espacio de Búsqueda y Restricciones Técnicas	17
3.2.3. Paisajes de Función Objetivo (Landscapes)	18
3.3. Software y Librerías	19
4. Metodología	23
4.1. Modelización del Paisaje y Codificación del Espacio de Búsqueda	23
4.1.1. Codificación Cuántica y Distribución de Recursos	24
4.1.2. Diseño del Marco Experimental y Baseline Clásico	24
4.2. Algoritmo QITE-VQE Proyectivo	25
4.2.1. Fundamentos de la Evolución en Tiempo Imaginario	25

4.2.2.	Diseño del Ansatz y Optimización Libre de Gradientes . . .	25
4.3.	Simulación de Ruido y Decoherencia en la Era NISQ	26
4.3.1.	Simulación mediante Redes de Tensores	26
4.3.2.	Canales de Error y Asimetría de Puertas	26
4.4.	Infraestructura HPC y Paralelismo	27
4.5.	Evaluación Comparativa y Baselines	27
4.6.	Pseudocódigo del Algoritmo Implementado	28
5.	Validación y Pruebas	31
5.1.	Diseño de la Campaña Experimental	31
5.2.	Benchmarks de Escalabilidad y Precisión	32
5.2.1.	Resultados de Precisión General	32
5.2.2.	Estabilidad y Análisis de Outliers	32
5.3.	Pruebas de Resiliencia ante el Ruido	33
5.4.	Validación del Coste y Frontera de Eficiencia	34
6.	Discusión de los Resultados	37
6.1.	Verificación de las Hipótesis y Cierre de la Brecha Estructural . . .	37
6.2.	Comparativa con los Baselines del Estudio	38
6.2.1.	Frente al Baseline Cuántico: Standard-VQE	38
6.2.2.	Frente al Baseline Clásico: Optimización Bayesiana	38
6.3.	Implicaciones Transversales	39
6.3.1.	Aspectos Éticos y Medioambientales: Hacia una IA Verde .	39
6.3.2.	Aspectos Socioeconómicos: Democratización y Competiti- vidad	40
6.3.3.	Aspectos Legales y Culturales: El Nuevo Paradigma del QML	40
7.	Conclusiones y posibles ampliaciones	43
7.1.	Resumen de las Principales Aportaciones	43
7.2.	Suposiciones y Limitaciones del Estudio	44
7.3.	Vías de Mejora y Trabajo Futuro	45
A.	Manuales técnicos	53
A.1.	Estructura de ficheros	53
A.2.	Creación del entorno	54
A.3.	Reproducción de los resultados	55
B.	Licencia	57

Índice de figuras

2.1.	Figura del efecto del valor del learning rate sobre la pérdida . . .	10
2.2.	Figura sobre los diferentes usos de la computación cuántica	12
2.3.	Figura de la intersección de computación cuántica y <i>Machine Learning</i>	12
5.1.	Evolución de la precisión final en función del número de qubits. .	32
5.2.	Dispersión estadística de la precisión por número de qubits. . . .	33
5.3.	Degradación media ante ruido.	34
5.4.	Matriz detallada para QITE-VQE.	34
5.5.	Mapa de eficiencia de Pareto: Compromiso entre precisión alcanzada y tiempo consumido.	35
5.6.	Crecimiento logarítmico del tiempo de simulación.	35

Índice de tablas

3.1. Nodos de cómputo principales del clúster FinisTerra III (CESGA).	15
3.2. Crecimiento exponencial del tamaño en disco de los paisajes y filtros precalculados según la escala de qubits.	19

Capítulo 1

Introducción

Uno de los grandes problemas de las redes neuronales artificiales es la selección adecuada del valor de ciertos parámetros de la red, llamados hiperparámetros, que mejorarían su rendimiento. Estos parámetros no son aprendidos, por lo que tienen que fijarse previamente al entrenamiento de las redes neuronales artificiales. Los hiperparámetros pueden estar relacionados con la arquitectura o estructura del modelo (número de capas ocultas, función de activación, tamaño de un filtro de convolución...) o con el propio proceso de entrenamiento (tasa de aprendizaje, tamaño de lote...).

Antiguamente, los hiperparámetros se elegían de forma manual bajo el criterio de los expertos que diseñaban la red neuronal. Este sistema tenía la gran desventaja de que requería disponer de personal experto, lo que tardaba gran cantidad de tiempo si la red tiene un tamaño grande y además no asegura encontrar la configuración de hiperparámetros óptima. La tendencia es trabajar con arquitecturas cada vez más grandes y pesadas, es decir, con mayor número de hiperparámetros. Por ello se están usando cada vez más técnicas de aprendizaje automático (ML por sus siglas en inglés), puesto que ahorra el coste de personal a cambio de aumentar el coste computacional.

Un componente clave de ML, y el que será objeto de estudio en este trabajo, es la optimización de hiperparámetros (HPO por sus siglas en inglés). En este componente se abarcan numerosos algoritmos así como estrategias para obtener de manera automática las configuraciones de hiperparámetros óptimas. Por lo que se busca optimizar una función muy costosa de evaluar, ya que conocer su valor exacto para una configuración en concreto de hiperparámetros implica entrenar el modelo N épocas con dicha configuración. Es por esto que HPO es un proceso que, a menudo, tiene un elevado coste computacional.

Debido a que estos hiperparámetros tienen valores continuos, existen infinitos valores que se podrían seleccionar para cada hiperparámetro (por ejemplo, la tasa

de aprendizaje o el número de capas ocultas). Si bien existen ciertas heurísticas que limitan ese número de valores, este sigue siendo muy elevado, de modo que las técnicas de optimización que se pueden desarrollar para elegir los valores más adecuados tienen tiempos de computación muy elevados, lo cual hace que no sea práctica su aplicación.

1.1. Computación cuántica

Los algoritmos cuánticos parten de una base diferente de los algoritmos clásicos, ya que se benefician de los principios de la mecánica cuántica [1]. Para poder entender su funcionamiento en profundidad, conviene entender antes qué es la computación cuántica [2] y qué diferencia tiene de la clásica. La principal diferencia radica en las unidades de medida del ordenador: en computación clásica se opera con bits que se representan de forma binaria con 0 o 1, mientras que en computación cuántica se opera con qubits [3] cuya formulación es:

$$|\psi\rangle = \alpha_0 |0\rangle + \alpha_1 |1\rangle \quad (1.1)$$

También se puede formular el desglose de un qubit, ya que resulta más intuitivo para comprender la ventaja que otorga al transportar una mayor densidad de información:

$$\begin{aligned} |\psi\rangle &= \alpha_0 |0\rangle + \alpha_1 |1\rangle \\ &= \alpha_0 \frac{|+\rangle + |-\rangle}{\sqrt{2}} + \alpha_1 \frac{|+\rangle - |-\rangle}{\sqrt{2}} \\ &= \frac{\alpha_0 + \alpha_1}{\sqrt{2}} |+\rangle + \frac{\alpha_0 - \alpha_1}{\sqrt{2}} |-\rangle \end{aligned} \quad (1.2)$$

Donde α_0 y α_1 son números complejos denominados *amplitudes de probabilidad*. Estos coeficientes deben satisfacer la condición de normalización $|\alpha_0|^2 + |\alpha_1|^2 = 1$, la cual garantiza que la suma de las probabilidades de obtener los estados $|0\rangle$ o $|1\rangle$ al realizar una medición sea la unidad. Esta propiedad, conocida como la *Regla de Born*, es la que permite que un único qubit transporte una densidad de información significativamente superior a la de un bit clásico antes del colapso de su estado [4].

Los qubits son similares en el sentido de representarse también con 0 o 1, pero a diferencia de los bits clásicos, pueden encontrarse en un estado de superposición coherente, cuya fórmula general para un sistema de n qubits es:

$$|\psi\rangle = \alpha_{00\dots 0} |00\dots 0\rangle + \alpha_{00\dots 1} |00\dots 1\rangle + \dots + \alpha_{11\dots 1} |11\dots 1\rangle \quad (1.3)$$

Donde los coeficientes $\alpha_{00\dots 0}, \alpha_{00\dots 1}, \dots, \alpha_{11\dots 1}$ son las amplitudes de probabilidad y pueden tomar valores complejos.

Un ordenador clásico tendría que realizar operaciones secuenciales si quisiera evaluar los estados 0 y 1, mientras que un ordenador cuántico, gracias a la superposición, puede operar sobre ambos estados simultáneamente en una única operación. Como se puede intuir, esto representa una gran ventaja en términos de velocidad de cómputo. Cabe aclarar que un qubit es un sistema cuántico de dos niveles que existe en una superposición de sus estados base ($|0\rangle$ y $|1\rangle$). Al realizar una medición para determinar el estado resultante, la superposición colapsa de forma irreversible en un único estado clásico definido, bien sea 0 o 1.

Otro elemento importante y diferente en la computación cuántica, es el uso de puertas lógicas. Como es bien sabido, los ordenadores clásicos operan con semiconductores CMOS (complementary metal-oxide-semiconductor en inglés) [5]. Estos operadores permiten hacer todo tipo de operaciones, por lo que son muy versátiles, en cambio, en computación cuántica hay operaciones que por lógica matemática son imposibles de hacer, lo cual es una limitación y una desventaja en contra de la computación cuántica. Es por ello que las puertas Pauli-X (equivalente al NOT clásico, para 1 qubit) o también llamadas CNOT (controlled-NOT en inglés, para 2 qubits) [6] son fundamentales y de las más usadas, al hacer una de las operaciones más básicas e importantes de todas, la operación NOT. Esta es su formulación y representación:

$$\text{CNOT: } \begin{array}{c} A \text{---} \bullet \\ | \\ B \text{---} \oplus \end{array} \quad \text{CNOT}_{AB} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad (1.4)$$

Esta puerta lógica es la base de muchas otras ya que por ejemplo, para hacer un SWAP, se puede hacer con 3 puertas CNOT, tal que:

$$\text{SWAP : } \begin{array}{c} A \text{---} \times \\ | \\ B \text{---} \times \end{array} \sim \begin{array}{c} \bullet \quad \oplus \quad \bullet \\ | \quad | \quad | \\ \oplus \quad \bullet \quad \oplus \end{array} \sim \begin{array}{c} \oplus \quad \bullet \quad \oplus \\ | \quad | \quad | \\ \bullet \quad \oplus \quad \bullet \end{array} \quad (1.5)$$

o una puerta c-Z (Pauli-Z) se haría con 2 puertas Hadamard y 1 CNOT, de esta forma:

$$c-Z : \begin{array}{c} A \text{---} \bullet \\ | \\ B \text{---} \bullet \end{array} \sim \begin{array}{c} \boxed{H} \quad \oplus \quad \boxed{H} \\ | \quad | \quad | \\ \bullet \quad \bullet \quad \bullet \end{array} \sim \begin{array}{c} \bullet \\ | \\ \boxed{H} \quad \oplus \quad \boxed{H} \end{array} \quad (1.6)$$

Las puertas CNOT además de para crear los circuitos lógicos y realizar las operaciones, tienen también otra función importante, ya que sirve como unidad de medida en el tamaño de los circuitos. Dado el ruido que conlleva añadir cada puerta, es evidente que se buscarán crear circuitos con el menor número de puertas CNOT posible, es decir de menor tamaño. Cuanto más ruido haya, menor

será la tasa de éxito que se obtenga al ejecutar en un computador cuántico, la fórmula de la Probabilidad de Éxito Estimada (ESP) [7] sería la siguiente:

$$ESP = \prod_{i=1}^{N_{total}} (1 - \epsilon_i) \quad (1.7)$$

En la cual ϵ_i es el error de cada puerta. Se puede representar de esta otra forma la fórmula:

$$P_{exito} \approx (1 - \epsilon_{CNOT})^{N_{CNOT}} \quad (1.8)$$

En donde como se puede ver, a mayor número de puertas CNOT, menor será la tasa de éxito de la ejecución. Partiendo de la anterior fórmula, existe esta que puede llegar a ser más intuitiva aún:

$$P_{exito} \approx e^{-N_{CNOT} \cdot \epsilon_{CNOT}} \quad (1.9)$$

1.2. Algoritmos cuánticos

Una vez comprendido el funcionamiento básico de la computación cuántica, se puede intuir que tipo de ventajas aportarán los algoritmos cuánticos frente a los clásicos. Para comprender mejor las ventajas que pueden aportar, se debe primero entender su funcionamiento, y en qué difieren de los algoritmos clásicos. Como era de esperar, la idea principal de los algoritmos cuánticos es que se aprovechen de los principios cuánticos y gracias a la superposición puedan llegar a soluciones óptimas en menor tiempo que los algoritmos clásicos [8].

Hay que hacer dos grandes distinciones en los algoritmos de optimización investigados, un grupo son los algoritmos cuántico-inspirados [9], [10], los cuales no se pueden ejecutar en ordenadores cuánticos, ni generan circuitos con puertas CNOT. Y por el otro lado, están los algoritmos cuánticos, que si se pueden ejecutar en ordenadores cuánticos. Los algoritmos cuántico-inspirados QIAOs (Quantum-Inspired Optimization Algorithms), son muy útiles ya que presentan ventajas de optimalidad y velocidad frente a los algoritmos clásicos, pese a estar ejecutados ambos en hardware clásico.

Este estudio se centrará sobre todo en el segundo tipo de algoritmos, los cuánticos reales. Aunque cabe destacar la eficacia de los cuántico-inspirados como: QPSO (Quantum-behaved Particle Swarm Optimization, por sus siglas en inglés) [11], [12], QACO (Quantum Ant Colony Optimization, por sus siglas en inglés) [13], [14] o QGA (Quantum Genetic Algorithm, por sus siglas en inglés) [15], [16]. Estos 3 algoritmos funcionan como metaheurísticas ya que obtienen una precisión alta, no óptima, en un tiempo muy bajo a comparación del resto de algoritmos.

En los algoritmos cuánticos se pueden diferenciar varias familias. La más grande de todas es la de algoritmos variacionales VQA donde se encuentran VQE y sus variantes [17], [18], así como QAOA y sus variantes [19]-[23], además de optimizadores sin gradiente como pueden ser FALQON [24], [25], Rotosolve y sus variantes [26], [27], así como FRAXIS [28], [29]. La tercera familia es la de algoritmos de búsqueda cuántica en donde están Grover y sus variantes [30], [31], QA (Quantum Annealing, por sus siglas en inglés) [32], y NV-QWOA (Non-Variational Quantum Walk Optimization Algorithm, por sus siglas en inglés) [33], [34]. La penúltima familia es la de búsqueda de arquitecturas cuánticas donde se encuentran algoritmos como DQAS y sus variantes [35]-[37], así como EQNAS [38] y QuantumNAS [39]. Por último, cabe mencionar a la familia del aprendizaje cuántico (QML), en donde hay algoritmos como Q-GP-UCB (Quantum-Gaussian Process UCB, por sus siglas en inglés), QSVR [40], [41], VQC-RL [42] entre otros varios.

1.3. Objetivos del trabajo

Tras los avances de la computación cuántica junto con los algoritmos cuánticos, se generó una duda, y es si sería posible optimizar los hiperparámetros (HPO) de los modelos usando los paradigmas y beneficios de la computación cuántica. Justamente uno de los objetivos de este trabajo es hacer un estudio de las aproximaciones actuales para la optimización de hiperparámetros en el campo del aprendizaje automático, en general, y en el de las redes neuronales, en particular de Transformers. Además de llegar a seleccionar la combinación óptima del conjunto de hiperparámetros a mejorar.

Una vez elegida la estrategia óptima, se requería el objetivo de diseñar e implementar una solución basada en computación cuántica para la optimización de hiperparámetros en redes neuronales artificiales, concretamente, en redes neuronales de propagación hacia adelante (feed-forward). Por lo tanto, el último objetivo es realizar una comparación del rendimiento de la solución basada en computación cuántica con otras soluciones de optimización de hiperparámetros.

La comparación se hará con el fin de poder estudiar la viabilidad o no de estos algoritmos. Siendo así la finalidad, llegar a implementar y experimentar con algoritmos cuánticos con el fin de evaluar críticamente la viabilidad y el rendimiento de estas técnicas en la era NISQ [Noisy Intermediate-Scale Quantum], identificando tanto sus ventajas potenciales como sus limitaciones actuales frente a los métodos clásicos.

1.4. Implicaciones Transversales del Estudio

El desarrollo de soluciones híbridas cuántico-clásicas para la optimización de hiperparámetros trasciende el marco estrictamente técnico de la ingeniería de software, entrelazándose con repercusiones éticas, socioeconómicas, legales y culturales que contextualizan la relevancia de esta investigación en la sociedad tecnológica actual.

En el plano **ético y medioambiental**, la optimización de modelos profundos como los Transformers mediante técnicas clásicas tradicionales se enfrenta a un problema de sostenibilidad debido a la elevada huella de carbono derivada del uso ininterrumpido de clústeres de GPUs. La introducción de algoritmos cuánticos variacionales permite abordar este desafío bajo el paradigma de la Inteligencia Artificial Verde (*Green AI*). Al explotar el filtrado por tiempo imaginario, se optimiza la exploración del espacio de estados y se reduce drásticamente el número de entrenamientos iterativos clásicos necesarios, disminuyendo así el consumo energético global del proceso.

Desde una perspectiva **socioeconómica**, esta aproximación favorece la democratización de la Inteligencia Artificial de alta escala. Al reducir los costes computacionales asociados al diseño de redes neuronales, se mitiga el oligopolio de las grandes corporaciones, permitiendo que universidades, centros de investigación públicos y pequeñas o medianas empresas compitan en igualdad de condiciones. Asimismo, la validación de estos modelos en infraestructuras de acceso público, como el supercomputador FinisTerra III y el ordenador cuántico real Qmio del CESGA, subraya el valor estratégico de la inversión pública para garantizar la soberanía tecnológica y fomentar la retención de talento especializado.

Finalmente, en el ámbito **legal y cultural**, la automatización del diseño arquitectónico de software mediante el colapso de estados cuánticos probabilísticos plantea interrogantes inéditos en torno a la propiedad intelectual, la trazabilidad y la regulación de patentes para patrocinios algorítmicos no humanos. Culturalmente, este trabajo impulsa una transición fundamental en la formación técnica, promoviendo el paso desde la programación imperativa clásica hacia el desarrollo de algoritmia cuántica aplicada a problemas reales de MLOps.

1.5. Organización de la memoria

Esta memoria se estructura de la siguiente forma:

- En el **Capítulo 1** se introducen los conocimientos básicos sobre hiperparámetros, computación cuántica y algoritmos cuánticos.

- En el **Capítulo 2** se realiza una revisión del estado del conocimiento sobre optimización de hiperparámetros y la predicción de rendimiento y se analiza en profundidad el algoritmo analizado (QITE-VQE).
- En el **Capítulo 3** se introducen los materiales más relevantes utilizados durante el proyecto como el supercomputador FinisTerae III o el ordenador cuántico Qmio.
- En el **Capítulo 4** se describe la metodología seguida durante el proyecto y se introduce el algoritmo , ideado durante la realización de este trabajo.
- En el **Capítulo 5** se presentan las pruebas y resultados de los algoritmos cuánticos y clásicos junto con un análisis de los mismos, tanto en simulación cuántica como en ejecución real cuántica.
- En el **Capítulo 6** se extraen conclusiones y se proponen líneas de trabajo futuro.

Capítulo 2

Estado de conocimiento del problema a abordar

2.1. Estado del arte de HPO

2.1.1. Hiperparámetros relevantes

La optimización de hiperparámetros no es un problema exclusivo del ámbito del *machine learning* (ML), hay diferentes ámbitos además del aprendizaje automático profundo. Pero tal y como se avanzó en la introducción, esta área está adquiriendo gran relevancia y fuerza, con motivo de la inteligencia artificial. Justamente por ese objetivo de querer entrenar los mejores modelos y con menos errores posibles, los modelos están siendo entrenados con conjuntos de datos masivos (big data), aunque sin un ajuste de hiperparámetros correcto no se estaría exprimiendo el potencial total de un modelo.

Hay gran cantidad de hiperparámetros, por eso es clave la identificación de los más relevantes y de mayor impacto [43] en el entrenamiento de los modelos, ya que sería un coste demasiado elevado tanto temporal como computacional, tratar de ajustar todos los parámetros. Entre los hiperparámetros con mayor impacto está el batch size, que permite ajustar con que cantidad de datos se va a entrenar en el modelo, siendo este siempre múltiplo de 2. Este hiperparámetro además de influir en el aprendizaje del modelo, también es relevante en el tiempo de ejecución, ya que a medida que el valor aumenta, mayor será el tiempo de entrenamiento [44].

Otro hiperparámetro importante es el learning rate, ya que determina el tamaño del paso en el aprendizaje en el descenso de gradiente. Si es un valor demasiado grande no llegará a aprender y sobreajustará, y si es demasiado pequeño, se quedará estancado en óptimos locales. Es por eso fundamental elegir un valor intermedio óptimo para que pueda aprender en el modelo de manera correcta.

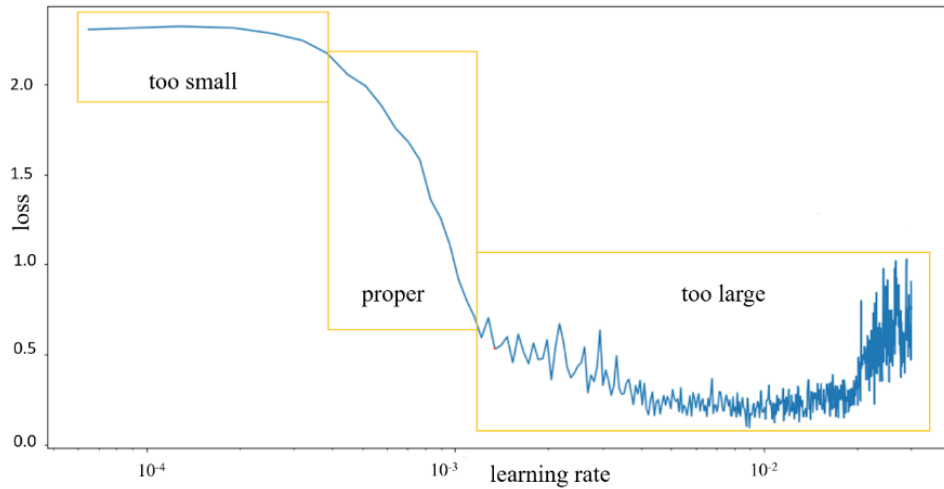


Figura 2.1: Figura del efecto del valor del learning rate sobre la pérdida

En este trabajo, el espacio de búsqueda se extiende a una arquitectura de tipo **Transformer**, incorporando parámetros estructurales de alta complejidad: el **EMB_SIZE** (tamaño del *embedding*), que define la riqueza de la representación interna; el **ATTN_HEADS**, que controla las cabezas de atención paralela; y el número de capas en el codificador y decodificador (**ENC/DEC_LAYERS**), que determinan la capacidad de abstracción de la red.

2.1.2. Algoritmos Clásicos de Optimización y Fine-Tuning

El proceso de *fine-tuning* o ajuste fino es la piedra angular del rendimiento en el aprendizaje profundo moderno. Debido a que un modelo no presenta resultados idénticos ante diferentes dominios de datos, la automatización de la búsqueda de hiperparámetros se vuelve obligatoria. Para resolver este problema, la literatura clásica ha evolucionado desde métodos heurísticos simples hacia estrategias probabilísticas avanzadas.

Búsqueda Exhaustiva y Aleatoria

- **Grid Search:** Realiza una búsqueda exhaustiva en una rejilla predefinida [45]. Aunque garantiza encontrar el mejor punto de la rejilla, padece de la maldición de la dimensionalidad y desperdicia recursos en dimensiones poco sensibles.
- **Random Search:** Propuesto como una alternativa más eficiente, selecciona configuraciones aleatoriamente. Su éxito radica en que permite explorar una mayor variedad de valores en las dimensiones críticas, superando habitualmente al Grid Search [45] con el mismo presupuesto computacional.

Optimización Bayesiana (BO)

La Optimización Bayesiana representa el estado del arte para la optimización de funciones de caja negra costosas de evaluar. A diferencia de los métodos anteriores, BO utiliza la información de evaluaciones pasadas para decidir de manera inteligente cuál será el próximo punto a probar [46]. Este proceso se basa en dos componentes fundamentales:

1. **Modelo Subrogado (Surrogate Model):** Dado que entrenar una red neuronal para evaluar una configuración es extremadamente costoso, BO construye un modelo estadístico que imita el comportamiento de la función real. En este trabajo, cobra especial relevancia el uso de **Random Forest** como modelo subrogado, el cual, mediante el uso de múltiples árboles de decisión, estima no solo el valor esperado de la precisión, sino también la incertidumbre en zonas no exploradas del espacio de búsqueda.
2. **Función de Adquisición:** Es la encargada de guiar la búsqueda. Utiliza la información del modelo subrogado para equilibrar la *exploración* (probar zonas con alta incertidumbre) y la *explotación* (probar zonas donde el modelo predice un buen rendimiento). Algoritmos como *Expected Improvement* (EI) permiten que BO converja hacia el óptimo global con un número de evaluaciones órdenes de magnitud menor que los métodos aleatorios.

Esta sofisticación en los métodos clásicos subrogados proporciona el marco comparativo ideal para este trabajo, donde se evaluará si las propiedades de la computación cuántica pueden ofrecer una ventaja competitiva sobre estos modelos probabilísticos.

2.2. Estado del arte en computación cuántica

La computación cuántica es una herramienta muy novedosa que está demostrando ser muy útil en diversas aplicaciones. Una de las más conocidas es en la farmacología, al ser capaz de computar simulaciones y descubrir nuevos fármacos en un menor tiempo. También en la rama médica al poder ser combinada con inteligencia artificial, y así poder detectar tumores o enfermedades de manera más precisa y veloz. En general, en el ámbito sanitario, químico y biológico, es extremadamente útil la computación cuántica, así como el QML (Quantum Machine Learning, por sus siglas en inglés) [47].

Otro ámbito muy relevante que cambió es la ciberseguridad, ya que los sistemas clásicos de seguridad, no estaban preparados para la potencia de cómputo ni los paradigmas de la computación cuántica. Esta área se ve afectada ya que los sistemas se tienen que modernizar para ser robustos incluso ante ataques cuánticos cibernéticos. Es por eso que se está empezando a hablar de "Quantum-IoT

12CAPÍTULO 2. ESTADO DE CONOCIMIENTO DEL PROBLEMA A ABORDAR

Security” [48], en donde se menciona que el mundo IoT en el que se vive actualmente necesita actualizarse y reforzar sus sistemas de encriptación y seguridad en general.

Hay usos más variados de la computación cuántica, como pueden ser procesamiento de imágenes, análisis de sentimientos, corrección de errores, *machine learning*... [49]

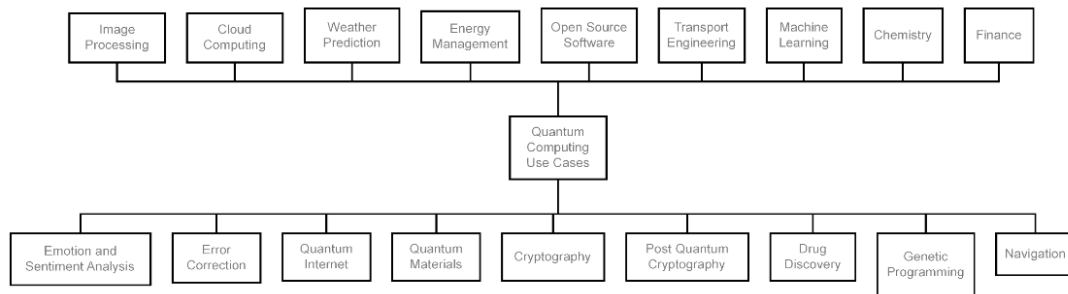


Figura 2.2: Figura sobre los diferentes usos de la computación cuántica

Este estudio realizado, se va a centrar en el *machine learning* cuántico, es decir, en el QML. El QML es precisamente procesar datos clásicos usando *machine learning* en ordenadores cuánticos. La idea clave es poder aprovechar las ventajas computacionales como: entrelazamiento, superposición, teorema de no clonación e interferencia. Esto ayuda a procesar información de forma más rápida, más allá de los límites clásicos.

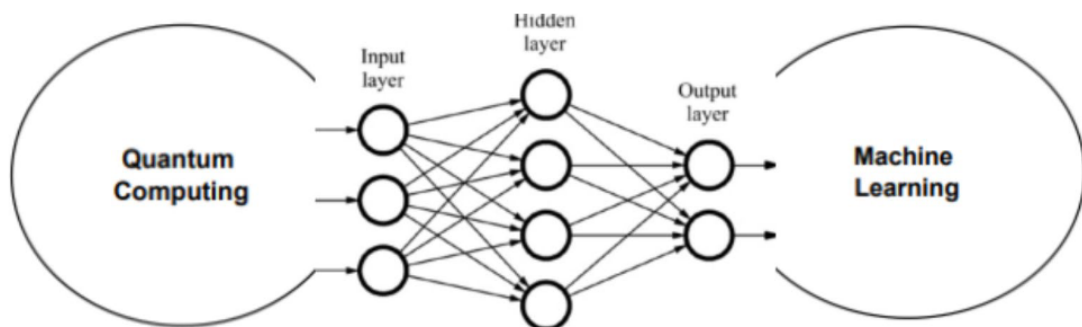


Figura 2.3: Figura de la intersección de computación cuántica y *Machine Learning*

2.2.1. QITE-VQE

Existen múltiples implementaciones cuánticas simples, las que transforman un algoritmo clásico en uno cuántico, véase $SVR \rightarrow QSVR$, o también $NN \rightarrow QNN$. Siendo así estas implementaciones más sencillas por poder partir de la base clásica. También hay implementaciones que no tienen su símil clásico, como serían algoritmos tipo QAOA, VQE (cuyos observables se evalúan mediante operadores de Pauli) o Grover, los cuales son algoritmos puramente cuánticos.

Una solución algo más compleja en cuanto a formulación matemática es el algoritmo QITE-VQE (Quantum Imaginary Time Evolution - Variational Quantum Eigensolver, por sus siglas en inglés) .

Este algoritmo usa de base un VQE que aprovecha la evolución en tiempo imaginario para amplificar la probabilidad del estado fundamental antes de calcular la función de coste, reduciendo por lo tanto su profundidad en el circuito que crea [50]. El tiempo imaginario que es lo que le da esa ventaja al baseline de VQE, se puede definir de la siguiente forma:

$$|\psi(\tau)\rangle = \frac{e^{-\tau\hat{H}}|\psi_0\rangle}{\|e^{-\tau\hat{H}}|\psi_0\rangle\|} \quad (2.1)$$

Siendo $|\psi\rangle$ el estado mientras que τ es el tiempo imaginario, además de que $\hat{H}$ es el operador hamiltoniano.

La potencia de esta formulación reside en que el operador $e^{-\tau\hat{H}}$ actúa como un **filtro cuántico**. En el contexto de HPO, el Hamiltoniano $\hat{H}$ codifica el paisaje de precisión de la red neuronal. Este filtro "suprime" las configuraciones de hiperparámetros con bajo rendimiento (estados de alta energía) y amplifica las zonas óptimas.

Desde un punto de vista técnico, esta guía hacia la solución permite alcanzar el estado fundamental con una **menor profundidad de circuito** que un VQE tradicional. En la actual era **NISQ** (*Noisy Intermediate-Scale Quantum*), reducir la profundidad es vital: cada capa adicional y cada puerta **CNOT** introducen ruido térmico y errores de puerta que degradan la fidelidad.

Al minimizar estas métricas, el algoritmo QITE-VQE aumenta las probabilidades de éxito en ejecuciones sobre hardware real como el chip *Qmio*.

2.2.2. Posicionamiento del Trabajo

A diferencia de los estudios que se limitan a comparar precisiones finales, este trabajo busca llenar un hueco en la literatura evaluando la **viabilidad estructu-**

ral de los algoritmos. Al reportar el número de puertas CNOT y la profundidad necesaria para cada escala (de 5 a 32 qubits) así como cada nivel de ruido (0 a 0.03), este estudio establece una relación directa entre la complejidad del hardware cuántico disponible y el beneficio obtenido en la optimización de modelos de inteligencia artificial de última generación.

Otra diferenciación clave de este trabajo es el paso a la práctica, ya que se consigue ejecutar y validar las hipótesis, es decir los resultados obtenidos en simulación, con ejecuciones en un ordenador cuántico real. Obteniendo así un estudio de mayor peso que no se queda simplemente en teorías y simulaciones, sino en simulaciones que luego se verifican en la realidad.

Capítulo 3

Materiales

En este capítulo se detallan los distintos recursos: hardware, conjuntos de datos y librerías de software, utilizados para el desarrollo del trabajo.

3.1. Recursos Hardware

3.1.1. Clúster de Supercomputación FinisTerra III (CESGA)

La totalidad de los experimentos de escalabilidad de este trabajo han sido ejecutados en el **clúster FinisTerra III**, gestionado por el Centro de Supercomputación de Galicia (CESGA). Este sistema de cómputo de altas prestaciones pertenece a la Red Española de Supercomputación (RES) y está disponible para proyectos de investigación universitaria.

El FinisTerra III cuenta con una arquitectura heterogénea compuesta por distintos tipos de nodos de cómputo, cuyas características se resumen en el Cuadro 3.1.

Tipo de Nodo	#Nodos	CPU	#Cores	Memoria	GPU	Red
Estándar (Ice Lake)	~462	icelake	64	256 GB	—	InfiniBand
GPU (A100)	~66	icelake	64	256 GB	Nvidia A100	InfiniBand
Fat/SMP (High-Mem)	20	Intel Xeon	64	2 TB	—	InfiniBand
Optane	1	Intel Xeon	96	8 TB	—	n/a

Tabla 3.1: Nodos de cómputo principales del clúster FinisTerra III (CESGA).

Para la ejecución de este trabajo se han utilizado de forma intensiva los **nodos Fat o SMP** (Symmetric Multi-Processing), que cuentan con 2 TB de memoria RAM por nodo. Estos nodos son imprescindibles para la simulación de circuitos

cuánticos de más de 25 qubits, ya que el vector de estado de un sistema de n qubits requiere almacenar 2^n amplitudes complejas, lo que supone un consumo de memoria de **64 GB para 32 qubits** usando precisión simple.

La planificación y gestión de los trabajos se realiza mediante el gestor de colas **SLURM** (Simple Linux Utility for Resource Management), accediendo a la partición medium con un tiempo máximo de ejecución de **72 horas por trabajo**.

Para maximizar el rendimiento de los experimentos, se ha diseñado una estrategia de paralelismo a dos niveles:

- Paralelismo entre nodos: Mediante el lanzamiento de hasta 35 trabajos SLURM independientes simultáneamente (cada una de las siete escalas de qubits en 5 semillas diferentes), distribuyendo los experimentos por semilla y escala.
- Paralelismo dentro del nodo: Usando ProcessPoolExecutor de Python para ejecutar sobre 20 simulaciones cuánticas en paralelo dentro de cada nodo Fat.

3.1.2. Ordenador Cuántico Qmio

Para evaluar la viabilidad en hardware real y trascender la simulación numérica, se ha dispuesto de acceso a **Qmio**, el ecosistema de computación cuántica alojado en el CESGA y operativo para la comunidad investigadora desde 2024.

Este sistema híbrido (HPCQC) se encuentra directamente integrado con el supercomputador FinisTerae III mediante una red de interconexión de latencia ultra-baja.

Las características técnicas de Qmio que fundamentan el diseño experimental de este trabajo son las siguientes:

- **Unidad de Procesamiento Cuántico (QPU):** Qmio integra un procesador de 32 qubits superconductores basados en la arquitectura *coaxmon*. Estos dispositivos operan como qubits de tipo *transmon* con frecuencias fijas situadas en el rango de 4 a 6 GHz. El diseño destaca por emplear una topología tridimensional para el control y la lectura, lo que permite minimizar el impacto de la diafonía (*crosstalk*) entre componentes.
- **Operaciones Nativas:** La infraestructura de control orquesta pulsos de microondas de alta precisión hacia la QPU. Esta capacidad resulta fundamental para el cálculo de los valores esperados de la energía, ya que el

paisaje de optimización se descompone en términos de matrices de Pauli que son procesadas de forma nativa por el hardware.

- **Nodos de Emulación de Alta Eficiencia:** El ecosistema incorpora nodos de cómputo especializados equipados con procesadores ARM A64FX. Estas unidades están optimizadas para la simulación cuántica de alto rendimiento, permitiendo validar técnicamente circuitos de hasta 34 qubits mediante librerías como `mpiQULACS` antes de proceder a la ejecución final sobre la QPU superconductora.

3.1.3. Hardware de Desarrollo Local

Para el desarrollo, pruebas y análisis de resultados se ha utilizado un ordenador personal con sistema operativo Windows 11, con acceso a los ficheros del clúster mediante protocolos SSH y SFTP para la transferencia de código y datos.

3.2. Conjuntos de datos

3.2.1. Oráculo de Rendimiento y Paisaje de Optimización Subrogado

Para evaluar la eficacia de los algoritmos de optimización de manera rigurosa, este trabajo emplea un enfoque de **Paisaje de Optimización Subrogado**. En lugar de realizar un entrenamiento real de la red neuronal en cada paso del algoritmo cuántico —lo cual sería computacionalmente inasumible—, se utiliza un oráculo que predice la precisión del modelo instantáneamente.

El núcleo de este oráculo es el dataset `datos_hpo_15q.json`, el cual contiene los resultados reales de un estudio exhaustivo sobre una arquitectura **Transformer**. Este dataset cubre un espacio de **15 qubits** (32,768 configuraciones únicas).

A partir de estos datos, se entrena un modelo de **Random Forest Regressor** que actúa como el "paisaje" de energía para el algoritmo cuántico. Este modelo no solo permite recuperar las precisiones del dataset original, sino que, mediante técnicas de interpolación y extrapolación, permite escalar el estudio de forma sintética pero realista hasta los **32 qubits**, manteniendo la estructura de correlaciones del problema original de HPO.

3.2.2. Espacio de Búsqueda y Restricciones Técnicas

El espacio de búsqueda se define en `hpo_spaces.py` y escala dinámicamente con el número de qubits disponibles. Para los experimentos de alta escala, se

optimizan los **6 hiperparámetros** críticos de la arquitectura Transformer:

1. `BATCH_SIZE`: Tamaño del lote de entrenamiento.
2. `LEARNING_RATE`: Tasa de aprendizaje del optimizador.
3. `EMB_SIZE`: Dimensión del modelo (*embedding size*).
4. `ATTN_HEADS`: Número de cabezas de atención.
5. `ENC_LAYERS`: Número de capas del codificador.
6. `DEC_LAYERS`: Número de capas del decodificador.

Una característica fundamental de este conjunto de datos es la presencia de una **restricción técnica de integridad**: la dimensión del modelo (`EMB_SIZE`) debe ser obligatoriamente divisible por el número de cabezas de atención (`ATTN_HEADS`). En el código de generación del paisaje (`generador_subrogado.py`), las configuraciones que no cumplen esta condición (`EMB_SIZE % ATTN_HEADS != 0`) son penalizadas con un valor de precisión nulo o negativo, creando “zonas prohibidas” en el paisaje de optimización que el algoritmo QITE-VQE debe aprender a evitar.

3.2.3. Paisajes de Función Objetivo (Landscapes)

Para posibilitar el escalado hasta los **32 qubits** ($2^{32} \approx 4,3 \times 10^9$ configuraciones), se entrena un modelo de regresión basado en **Random Forest** con 100 estimadores sobre los datos de 15 qubits. Este modelo es capaz de generalizar y predecir la precisión para cualquier punto del espacio hiperdimensional, permitiendo generar los *landscapes* binarios que sirven como Hamiltoniano del problema.

Los paisajes resultantes se almacenan en formato `.npy` y se normalizan en el rango $[-1, 0]$, donde -1 representa la máxima precisión (objetivo de minimización para el algoritmo cuántico). Gracias al uso de **Memory-Mapping**, es posible procesar estos paisajes (que alcanzan los **17 GB** para 32 qubits) de forma eficiente en los nodos *Fat* del CESGA.

Para hacer viables las simulaciones a gran escala, el trabajo precalcula y almacena en disco el **vector de función objetivo** (o *landscape*) para cada escala de qubits. Este vector asigna a cada posición binaria (configuración de hiperparámetros) su valor de precisión estimado mediante un modelo subrogado de tipo Random Forest (`generador_subrogado.py`).

Los *landscapes* se almacenan en formato `.npy` (NumPy) utilizando **Memory-Mapping** (`mmap_mode='r'`), lo que permite compartir el mismo fichero en disco

entre todos los procesos paralelos sin duplicar su contenido en RAM, reduciendo el consumo de memoria en un factor igual al número de procesos simultáneos. El tamaño de estos ficheros crece exponencialmente con el número de qubits:

Escala	Nº Configuraciones (2^n)	Tamaño Landscape (.npy)	Tamaño Filtro QITE (.npy)
5q	32	~128 B	~128 B
10q	1.024	~4 KB	~4 KB
15q	32.768	~128 KB	~128 KB
20q	1.048.576	~4 MB	~4 MB
25q	33.554.432	~128 MB	~128 MB
30q	1.073.741.824	~4 GB	~4 GB
32q	4.294.967.296	~17,2 GB	~17,2 GB

Tabla 3.2: Crecimiento exponencial del tamaño en disco de los paisajes y filtros precalculados según la escala de qubits.

3.3. Software y Librerías

El lenguaje de programación empleado en todo el trabajo es **Python 3.10**, por su amplio ecosistema de bibliotecas científicas, su compatibilidad con los módulos de simulación cuántica y su uso generalizado en el ámbito de la investigación en inteligencia artificial y computación cuántica. A continuación se describen las bibliotecas más relevantes.

Qiskit Es el *framework* de computación cuántica de código abierto desarrollado por IBM. En este trabajo se utilizan dos de sus módulos principales:

- `qiskit.circuit.library.EfficientSU2`: Proporciona el ***ansatz* variacional** basado en capas de puertas R_Y y R_Z con **entrelazamiento lineal**, que sirve como circuito parametrizado para los algoritmos VQE.
- `qiskit.transpile`: Se utiliza para compilar y **optimizar el circuito cuántico** para el *backend* de simulación específico, reduciendo el número de operaciones.

Qiskit Aer Es el simulador de alto rendimiento del ecosistema Qiskit. Permite simular la ejecución de circuitos cuánticos en *hardware* clásico mediante distintos métodos. En este trabajo, para garantizar la consistencia metodológica y la eficiencia computacional, se han seleccionado dos estrategias:

- **statevector**: Simulación exacta del vector de estado. Se utiliza exclusivamente para todos los experimentos en condiciones de **ausencia de ruido** (Entorno Ideal, Noise = 0.0), proporcionando el límite superior de precisión teórica.

- `matrix_product_state` (MPS): Simulación basada en **redes de tensores**. Se emplea de forma unificada para **todos los experimentos con ruido** (desde 5 hasta 32 qubits). Este método permite manejar la complejidad introducida por el modelo de ruido de forma eficiente, aprovechando la estructura de bajo entrelazamiento de los circuitos variacionales lineales empleados.

El modelo de ruido se implementa mediante `NoiseModel` con **errores despolarizantes**: un error de probabilidad p para puertas de un qubit y $10p$ para puertas de dos qubits (CX), modelando el comportamiento realista del *hardware* cuántico actual. Los niveles de ruido explorados son $p \in \{0.0, 0.01, 0.02, 0.03\}$

NumPy Es la biblioteca fundamental para computación numérica en Python. En este trabajo se utiliza intensivamente para:

- Almacenamiento y manipulación eficiente de los **vectores de estado y landscapes** mediante *arrays* de precisión simple (`float32`).
- Cálculo **vectorizado** de la función de coste QITE mediante operaciones *in-place* para minimizar el uso de memoria.
- Lectura de ficheros `.npy` mediante **Memory-Mapping**, que permite acceder a *arrays* de varios GB sin cargarlos completamente en RAM.

SciPy Proporciona el **optimizador clásico** utilizado en el bucle variacional. Concretamente, se emplea el método **COBYLA** (*Constrained Optimization BY Linear Approximations*) de `scipy.optimize.minimize`, que no requiere el cálculo del gradiente de la función de coste y resulta adecuado para el **entorno ruidoso** de la simulación cuántica. Se configura con un máximo de 150 iteraciones por experimento.

scikit-learn (*Machine Learning*) Se utiliza para el componente de aprendizaje automático del trabajo. Concretamente, para entrenar el **modelo subrogado** (`generador_subrogado.py`) que aproxima la función objetivo del HPO. Se emplea la clase `RandomForestRegressor` con 100 estimadores. Además, se utilizan utilidades de preprocesamiento para escalar los datos y codificar las variables categóricas del espacio de búsqueda.

Concurrent.futures (**Paralelismo Clásico**) Para aprovechar la arquitectura de **64 núcleos** de los nodos *Fat* del FinisTerra III, se utiliza la clase `ProcessPoolExecutor` de la biblioteca estándar `concurrent.futures`. Esta librería permite la **ejecución asíncrona** de múltiples instancias del simulador cuántico en procesos independientes, superando las limitaciones del *Global Interpreter Lock* (GIL) de Python y permitiendo un **escalado lineal** del rendimiento dentro de cada nodo.

Gestión de Datos y Visualización (Pandas, Matplotlib, Seaborn) Para el análisis y la representación de los resultados se emplean:

- **Pandas:** Fundamental para la carga, limpieza y unificación de los múltiples ficheros CSV generados por los nodos del clúster. Permite realizar operaciones de **agregación estadística** (medias, desviaciones típicas) por escala y algoritmo.
- **Matplotlib y Seaborn:** Utilizadas para generar las figuras de la memoria, incluyendo **gráficos de escalabilidad**, curvas de convergencia y análisis de estabilidad frente al ruido.

Infraestructura y CLI (Argparse, JSON, CSV) El flujo de trabajo se apoya en librerías de sistema para la comunicación con el clúster:

- **argparse:** Para la creación de la **interfaz de línea de comandos** de `runner_final.py`.
- **json y csv:** Para la lectura de *datasets* y **persistencia de resultados**.

SLURM (Sistema de Gestión de Recursos) Si bien no es una biblioteca de Python, el gestor de colas SLURM es un componente fundamental de la infraestructura. Se han desarrollado *scripts* Bash específicos para orquestar el **lanzamiento masivo y paralelo** de experimentos en el clúster, incluyendo soporte para el filtrado por semilla (`--seeds`) que permite **distribuir la carga de trabajo** de forma granular entre los nodos disponibles.

Capítulo 4

Metodología

En este capítulo se detalla el marco metodológico diseñado para la optimización de hiperparámetros (HPO) mediante computación cuántica variacional. La investigación se ha estructurado en un flujo de trabajo integral que abarca desde la modelización de paisajes subrogados de alta fidelidad hasta la ejecución distribuida en infraestructuras de supercomputación. El objetivo principal es proporcionar un entorno de experimentación robusto que permita evaluar la escalabilidad del algoritmo QITE-VQE Proyectivo en sistemas de hasta 32 qubits bajo condiciones realistas de ruido NISQ, así como estudiar empíricamente su viabilidad frente a técnicas clásicas consolidadas.

4.1. Modelización del Paisaje y Codificación del Espacio de Búsqueda

Uno de los pilares centrales de la metodología es el diseño de un **Modelo Subrogado** de alta fidelidad para simular el rendimiento de arquitecturas Transformer. Se ha adoptado esta aproximación por considerarla absolutamente indispensable, dado que el coste computacional de entrenar miles de modelos clásicos de gran tamaño en cada paso de la optimización cuántica resultaría prohibitivo.

Para construir este paisaje, se ha partido de un dataset empírico de *Process Mining* (`env_permit`) donde se evalúa la precisión de un Transformer sobre un espacio inicial de 15 qubits. Con el fin de extender este estudio de forma realista hasta el límite de los 32 qubits, se ha implementado un sistema de extrapolación topológica basado en un regresor (*Random Forest Regressor*) con 100 estimadores independientes. Se ha validado este modelo obteniendo un coeficiente de determinación $R^2 = 0,969$, lo que garantiza que las predicciones de energía para nuevas configuraciones son altamente fiables respecto a la dinámica de convergencia real.

Dada la enorme dimensionalidad de la simulación, que alcanza los 2^{32} estados posibles, se ha desarrollado un componente denominado **SurrogateManager** para gestionar la eficiencia computacional. Mediante el uso de técnicas de mapeo de memoria (*memory-mapping*) y archivos binarios `.npy`, se ha logrado que múltiples procesos concurrentes en el supercomputador accedan al paisaje físico en disco sin duplicar vectores en la RAM. Este diseño ha permitido saturar los nodos de computación del CESGA sin riesgo de colapsos por desbordamiento de memoria.

4.1.1. Codificación Cuántica y Distribución de Recursos

Se ha formalizado la traducción de los hiperparámetros clásicos al dominio cuántico mediante un esquema de codificación binaria directa. Para gestionar variables con rangos tan dispares, se ha implementado un algoritmo de distribución dinámica de bits que asigna recursos según la criticidad de cada parámetro. El espacio de búsqueda $\mathcal{X}$ se descompone asignando bits específicos para el *Batch Size* (rango $[16, 1024]$), el *Learning Rate* (escala logarítmica $[10^{-2}, 10^{-6}]$) y la estructura de capas y cabezas de atención.

Matemáticamente, se reconstruye la configuración escalar $\vec{h}$ a partir de una cadena de bits x de longitud n operando mediante la siguiente transformación:

$$\vec{h}_i = \text{Space}_i \left[\sum_{j=0}^{k_i-1} x_j \cdot 2^j \right] \quad (4.1)$$

Para integrar este espacio en el algoritmo cuántico, se ha desarrollado un **Oráculo Estricto** que mapea los valores de precisión a un Hamiltoniano diagonal conformado por cadenas de operadores de **Pauli-Z**. Este oráculo incorpora penalizaciones estructurales severas (P_{max}) para suprimir configuraciones inválidas, como aquellas donde la dimensión de *embedding* no es divisible por las cabezas de atención:

$$\hat{H}_{penalty}(x) = \begin{cases} E(x) & \text{si EMB_SIZE} \equiv 0 \quad (\text{mód ATTN_HEADS}) \\ P_{max} & \text{en otro caso} \end{cases} \quad (4.2)$$

4.1.2. Diseño del Marco Experimental y Baseline Clásico

Para evaluar la viabilidad de la propuesta frente al estado del arte, se ha desplegado un marco experimental de gran escala. Se ha ejecutado un total de **5250 evaluaciones de modelos**, desglosadas en las 7 escalas de qubits estudiadas (de 5 a 32 qubits) y empleando 5 semillas aleatorias (1001, 123, 99, 42 y 7) por escala para garantizar la robustez estadística.

En cada ejecución, se ha fijado un presupuesto de 150 iteraciones. Este volumen de datos permite comparar QITE-VQE con la **Optimización Bayesiana**, la cual se ha configurado como un proceso intrínsecamente secuencial donde cada modelo aprende de los resultados de los anteriores. Este enfoque comparativo es fundamental en la metodología, ya que permite identificar el punto de ruptura donde el filtrado cuántico en el paisaje ofrece ventajas competitivas sobre el aprendizaje iterativo de los métodos clásicos.

4.2. Algoritmo QITE-VQE Proyectivo

El núcleo algorítmico central de la investigación radica en una implementación avanzada y estable del *Quantum Imaginary Time Evolution* (QITE), formulada explícitamente para destacar en el régimen variacional.

4.2.1. Fundamentos de la Evolución en Tiempo Imaginario

A diferencia de un solver VQE estándar, —el cual suele padecer problemas severos al quedar atrapado en mínimos locales estériles o planicies de gradiente (*barren plateaus*)—, el algoritmo desplegado utiliza las bondades matemáticas del operador de evolución no unitaria $e^{-\tau\hat{H}}$.

Para operarlo eficientemente sin incurrir en un sobre coste de circuito inasumible, se ha derivado una aproximación proyectiva en la cual el valor esperado de la energía se evalúa sobre una distribución de probabilidad que actúa como un poderoso filtro cuántico:

$$\langle E \rangle_{filt} = \frac{\sum_x P(x, \theta) \cdot E(x) \cdot e^{-2\tau E(x)}}{\sum_x P(x, \theta) \cdot e^{-2\tau E(x)}} \quad (4.3)$$

Este innovador enfoque proyectivo permite concentrar la función de coste y “enfocar” la búsqueda hacia el estado fundamental subyacente de forma casi determinista, retroalimentando la convergencia a medida que avanza la ventana de tiempo imaginario τ .

4.2.2. Diseño del Ansatz y Optimización Libre de Gradientes

Para la preparación inicial del estado cuántico de prueba, se ha empleado el ansatz de preservación de hardware **EfficientSU2**, fundamentado en la aplicación de rotaciones uniparamétricas R_y y R_z (generadas por las correspondientes matrices).

Para la gestión de la actualización de pesos se ha seleccionado **COBYLA** (*Constrained Optimization By Linear Approximation*). Esta decisión es algorítmicamente crítica, ya que al ser un optimizador clásico de orden cero, otorga la capacidad de lidiar con funciones de coste altamente ruidosas sin requerir el cálculo explícito y analítico de derivadas o gradientes, una operación que es prohibitivamente costosa y propensa al colapso por error estadístico (*shot noise*) en simulaciones profundas.

4.3. Simulación de Ruido y Decoherencia en la Era NISQ

Para dotar a esta investigación de valor práctico y asegurar que las métricas obtenidas sean directamente extrapolables al comportamiento en hardware físico real, se ha orquestado un modelo de ruido estocástico integral valiéndose del simulador Aer.

4.3.1. Simulación mediante Redes de Tensores

Dada la insalvable complejidad temporal y espacial que exige mantener en memoria un vector de estado de 32 qubits, las ejecuciones se han articulado mediante el uso de *Matrix Product States* (MPS). Esta formulación tensorial representa el estado cuántico como una cadena enlazada de tensores y ha permitido acometer los experimentos fijando un límite dinámico para la dimensión de enlace (*bond dimension*). Este ajuste resulta ideal, ya que preserva el entrelazamiento fundamental del ansatz HPO al tiempo que confina la complejidad computacional en un nivel operativo manejable.

4.3.2. Canales de Error y Asimetría de Puertas

El modelo de hardware ruidoso simula la disipación de información aplicando canales de despolarización estocástica (basados en errores) según una estricta jerarquía de fidelidad:

1. **Errores de 1 Qubit:** Proyectados de forma sistemática sobre todas las puertas de rotación local con una probabilidad de fallo basal p .
2. **Errores de 2 Qubits (CNOT):** Acoplados a las operaciones lógicas de entrelazamiento bidimensional con una probabilidad amplificada de un orden de magnitud $(10p)$.

Esta proporción de 1:10 refleja de manera fiel las especificaciones y calibraciones actuales de los procesadores superconductores NISQ de vanguardia, modelando correctamente las puertas CNOT como el cuello de botella central y principal vector de decoherencia lógica.

4.4. Infraestructura HPC y Paralelismo

El grueso de la experimentación masiva se ha llevado a cabo recurriendo a las capacidades del supercomputador **FinisTerra III** administrado por el CESGA. Para lidiar con la elevada concurrencia que demandan los barridos variacionales de COBYLA, se ha erigido una arquitectura propia de paralelismo extremo.

- **Estrategia de Workers y Afinidad de Hilos:** Se han utilizado las abstracciones nativas de `ProcessPoolExecutor` para saturar los 64 núcleos físicos de los nodos SMP disponibles. Internamente, cada nodo se divide en 8 trabajadores concurrentes, concediendo a cada uno exactamente 8 hilos de ejecución. Esto garantiza que las librerías algebraicas internas de `Qiskit` puedan ejecutar multiplicaciones de tensores de manera paralela sin entorpecer el Global Interpreter Lock (GIL) de Python.
- **Monitorización y Resiliencia Estructural:** Operar en un entorno de colas compartidas expone los experimentos a cortes e interrupciones. Por ello, el flujo de despliegue contempla un robusto mecanismo de tolerancia a fallos (*runner resume*) que asegura volcar y recuperar estados parciales para evadir los topes *wall-time* impuestos por `Slurm`, salvaguardando en todo momento la integridad transaccional de los datasets.

4.5. Evaluación Comparativa y Baselines

Para evaluar de forma rigurosa la viabilidad algorítmica y la competitividad de la propuesta cuántica frente a las técnicas consolidadas, se ha establecido un marco de comparación sistemático que se ejecuta transversalmente en todas las escalas de simulación (desde 5 hasta 32 qubits).

El rendimiento y la convergencia del algoritmo QITE-VQE Proyectivo se compara directamente frente a dos enfoques de referencia (*baselines*):

- **VQE Estándar:** Empleado como *baseline* cuántico para cuantificar de forma aislada la mejora que aporta el filtrado en tiempo imaginario respecto a la optimización variacional pura, midiendo específicamente la mitigación de los mínimos locales en paisajes no convexos.
- **Optimización Bayesiana (BO):** Seleccionada como *baseline* clásico por representar el estado del arte actual en la optimización de hiperparámetros (HPO).

Para garantizar la validez estadística de los resultados clásicos y estudiar la viabilidad cuántica frente al estado del arte clásico, se ha diseñado un despliegue

experimental masivo. Este proceso ha supuesto la ejecución de un total de 5250 entrenamientos de modelos Transformer, distribuidos estratégicamente para cubrir el espectro de escalabilidad propuesto. El diseño experimental se desglosa de la siguiente manera:

- **Escalas de Qubits:** Se han ejecutado pruebas en las 7 escalas definidas (5, 10, 15, 20, 25, 30 y 32 qubits), permitiendo observar el comportamiento de los algoritmos desde espacios de búsqueda reducidos hasta dimensiones de 2^{32} estados.
- **Robustez Estadística:** Para cada escala y algoritmo, se han empleado 5 semillas aleatorias diferentes (las mismas en cuántica que en clásica), mitigando el impacto del azar en la inicialización y garantizando la reproducibilidad.
- **Presupuesto de Optimización clásica:** Cada ejecución individual consta de 150 iteraciones (o 150 modelos Transformer). En el caso de la Optimización Bayesiana, este proceso es intrínsecamente secuencial: cada nuevo modelo 'aprende' de los resultados de los anteriores. Mediante la actualización constante del modelo subrogado y la función de adquisición, el algoritmo refina su búsqueda hacia las zonas de máxima precisión del paisaje.

Esta comparativa cruzada es fundamental dentro de la metodología, ya que permite estudiar empíricamente la viabilidad real de los algoritmos cuánticos en la era NISQ. Al enfrentar QITE-VQE Proyectivo contra la Optimización Bayesiana en diferentes magnitudes de ruido y dimensiones de búsqueda, se busca identificar en qué punto de escala algorítmica la aproximación cuántica logra estabilizarse y ofrecer una ventaja medible frente al mejor exponente de la computación clásica, analizando no solo la precisión final alcanzada, sino también la velocidad de convergencia.

4.6. Pseudocódigo del Algoritmo Implementado

El flujo algorítmico principal, orquestando el bucle del gradiente proyectivo, la interacción con el modelo NISQ y la extracción sistemática de la huella de hardware, se sintetiza en el siguiente bloque de pseudocódigo.

Algorithm 1: Algoritmo QITE-VQE Proyectivo para HPO

Entrada: Hamiltoniano Subrogado $\hat{H}$ (Oráculo), Tiempo total τ , Paso $\Delta\tau$, Ansatz $U(\theta)$

Inicialización:
 Estado inicial $|\psi(0)\rangle = U(\theta_0)|0\rangle$
 Configurar simulador tensorial MPS con canal de ruido asimétrico NISQ ϵ

/* Evolución Variacional Proyectiva en Tiempo Imaginario */
for $t = 0, \Delta\tau, 2\Delta\tau, \dots, \tau$ **do**
 1. Estimar distribución filtrada penalizada mediante evaluación de $\hat{H}_{penalty}$
 2. Calcular aproximación de la energía: $\langle E \rangle_{filt} = \frac{\sum_x P(x, \theta) \cdot E(x) \cdot e^{-2\Delta\tau E(x)}}{\sum_x P(x, \theta) \cdot e^{-2\Delta\tau E(x)}}$
 3. Resolver corrección de pesos mediante optimizador **COBYLA**:
 $\theta_{t+\Delta\tau} = \arg \min_{\theta} \langle E \rangle_{filt}$
 4. Aplicar rotaciones de Pauli (R_y, R_z) y compuertas lógicas:
 $|\psi(t + \Delta\tau)\rangle = U(\theta_{t+\Delta\tau})|0\rangle$

/* Extracción Analítica de Métricas de Hardware */
 $C_{final} \leftarrow$ Transpilar y descomponer $U(\theta_\tau)$ en las compuertas nativas del procesador
 $CNOTs \leftarrow$ Contabilizar volumetría de operaciones **cx** en C_{final}
 $Depth \leftarrow$ Calcular la profundidad crítica real del circuito ensamblado
 $Acc \leftarrow 1 - \langle \psi(\theta_\tau) | \hat{H} | \psi(\theta_\tau) \rangle$

Salida: $\vec{R} = \{Acc, CNOTs, Depth, Time_{total}\}$

Capítulo 5

Validación y Pruebas

En este capítulo se detallan los experimentos diseñados y ejecutados con el objetivo de evaluar el comportamiento, la estabilidad y la escalabilidad del algoritmo cuántico QITE-VQE implementado, así como su viabilidad en el ajuste de hiperparámetros en comparación con un baseline cuántico y un SOTA clásico.

Se describen los escenarios de prueba a los que ha sido sometido el modelo, así como los resultados cuantitativos obtenidos tanto en términos de precisión final como de resiliencia frente a modelos de ruido realista.

5.1. Diseño de la Campaña Experimental

La validación del presente trabajo se ha estructurado en una serie de pruebas de escalabilidad ejecutadas en su totalidad sobre la infraestructura del Supercomputador FinisTerae III (CESGA). Para asegurar la significancia estadística de los resultados y su reproducibilidad, cada experimento ha sido repetido utilizando un conjunto fijo de 5 semillas pseudoaleatorias predefinidas (7, 42, 99, 123 y 1001).

Las pruebas se han categorizado en dos grandes bloques:

- **Pruebas de Escalabilidad:** Evaluación de la precisión del algoritmo en un rango incremental que va desde un espacio de 5 qubits hasta los 32 qubits (simulando los mismos qubits que tiene el Qmio, ordenador cuántico del CESGA). Cubriendo así un espacio desde las 2^5 hasta las 2^{32} posibles combinaciones, haciendo por lo tanto un análisis sobre la escalabilidad en espacios masivos de conjuntos.
- **Pruebas de Resiliencia al Ruido:** Inyección controlada de ruido en el simulador para medir la degradación en hardware NISQ. Se evaluó primero en condiciones ideales (sin ruido) y luego incrementando el parámetro de ruido base hasta 0.03 (lo que equivale a un 3% de probabilidad de fallo en compuertas de 1 qubit y a un 30% en las compuertas CNOT de 2

qubits debido al factor de escala de 10x asumido), llevando la simulación al escenario realista más desfavorable.

5.2. Benchmarks de Escalabilidad y Precisión

5.2.1. Resultados de Precisión General

La Figura 5.1 muestra la evolución de la precisión lograda por los algoritmos analizados (Optimización bayesiana, QITE-VQE y VQE estándar) conforme crece la escala del problema. QITE-VQE muestra una estabilidad excepcional, consolidándose en torno al 80 % de precisión en todo el rango explorado, mientras que el método Standard-VQE acusa bajadas bruscas por inestabilidades en ciertas escalas superiores como en 20 qubits y en 32 qubits. Por otro lado, Optimización bayesiana también sufre una caída pronunciada en 20 qubits. Esta bajada pronunciada en 20 qubits se produce más concretamente en la semilla 42, pero más lejos de ser un outlier, deja claro la robustez del algoritmo QITE-VQE manteniendo la precisión en esas mismas condiciones, siendo por lo tanto mucho más estable y robusto, independientemente de la inicialización.

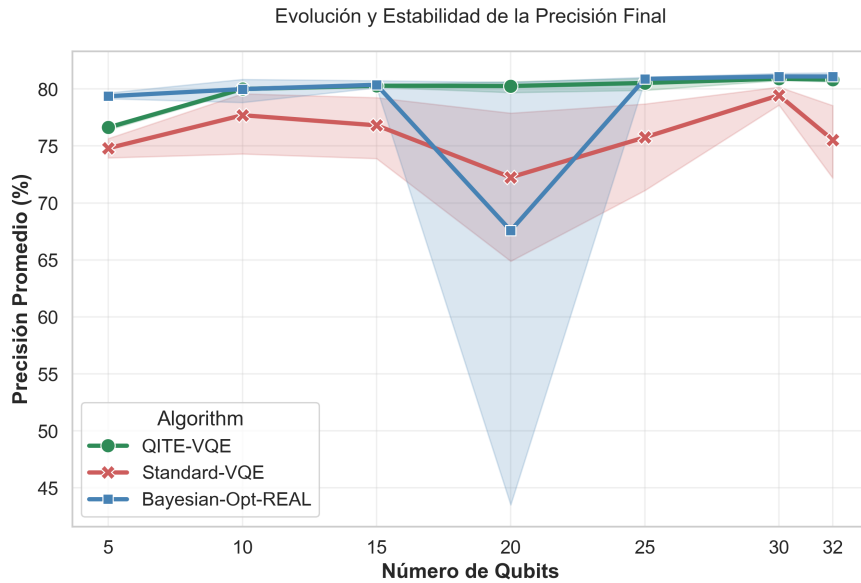


Figura 5.1: Evolución de la precisión final en función del número de qubits.

5.2.2. Estabilidad y Análisis de Outliers

El diagrama de caja segmentado por escala de la Figura 5.2 revela que, mientras los optimizadores clásicos y el VQE estándar presentan colas de fallo catastrófico considerables provocados por mínimos locales en escalas concretas, la

distribución de resultados de QITE-VQE es extremadamente compacta en todos los niveles de qubits, demostrando dominancia y consistencia en prácticamente todas las escalas.

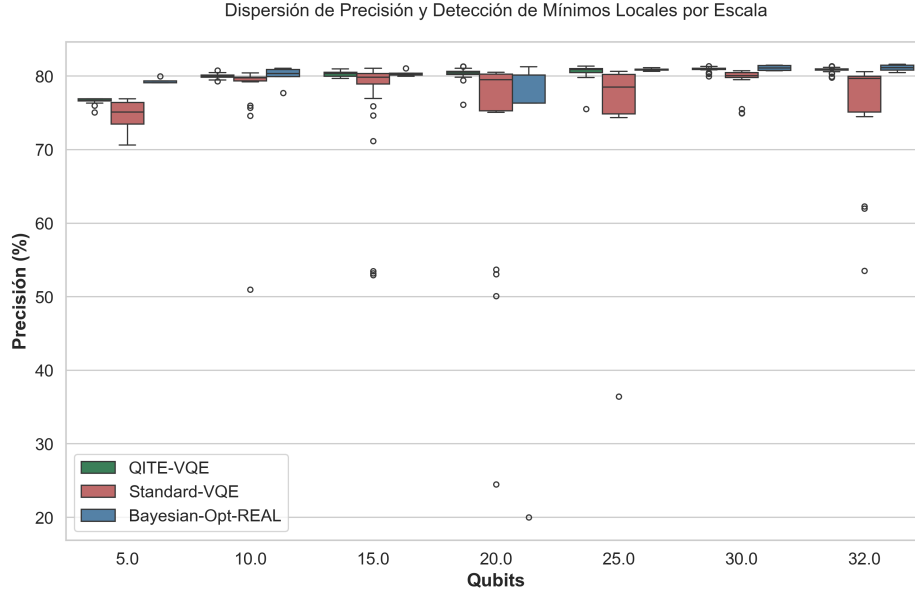


Figura 5.2: Dispersión estadística de la precisión por número de qubits.

5.3. Pruebas de Resiliencia ante el Ruido

En este experimento, al no haber ruido en computación clásica, se compara únicamente el *baseline cuántico* (VQE) con el algoritmo elegido (QITE-VQE). Como se evidencia en la Figura 5.3, el algoritmo QITE-VQE demuestra una estabilidad total, incluso siendo ligeramente mejor con ruido realista de 0.03 que sin ruido, en el escenario ideal. Por el contrario VQE estándar muestra nula resistencia al ruido, empeorando a medida que aumenta el ruido, y ampliando cada vez más su variabilidad, siendo por lo tanto muy poco estable en entornos realistas. Para observar el comportamiento parametrizado de QITE-VQE celda a celda, la Figura 5.4 expone una matriz cruzando escalas y niveles de ruido, validando visualmente el cuadrante de máxima fiabilidad operativa.

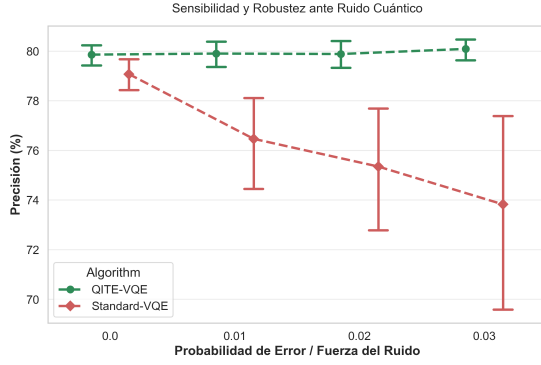


Figura 5.3: Degradación media ante ruido.

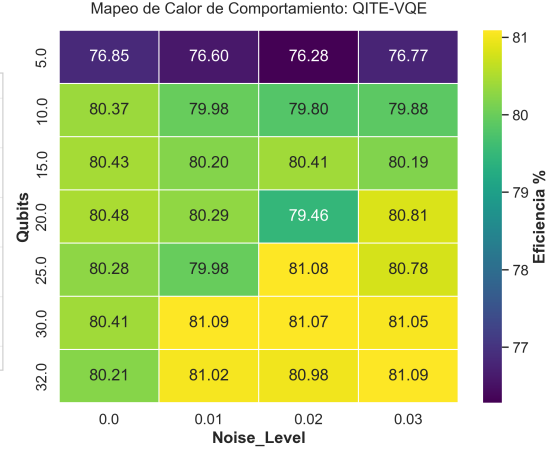


Figura 5.4: Matriz detallada para QITE-VQE.

5.4. Validación del Coste y Frontera de Eficiencia

Se ha construido la Frontera de Pareto Tiempo-Precisión (Figura 5.5). Esta gráfica sitúa las burbujas de QITE-VQE predominantemente en la franja superior de precisión operando en una fracción del tiempo en comparación con las búsquedas bayesianas clásicas y obteniendo resultados de mayor precisión que el baseline cuántico VQE estándar. Como se puede observar, las burbujas de QITE-VQE son superiores en precisión de manera absoluta a las de VQE estándar, además de ser más veloz requiriendo menor tiempo en relación con optimización bayesiana.

Destacando como se comentó anteriormente, la burbuja de 20 qubits en la que gracias a la semilla 42 se comprueba y reafirma la robustez de QITE-VQE mientras que para VQE estándar se baja la precisión y de forma aún más pronunciada en optimización bayesiana.

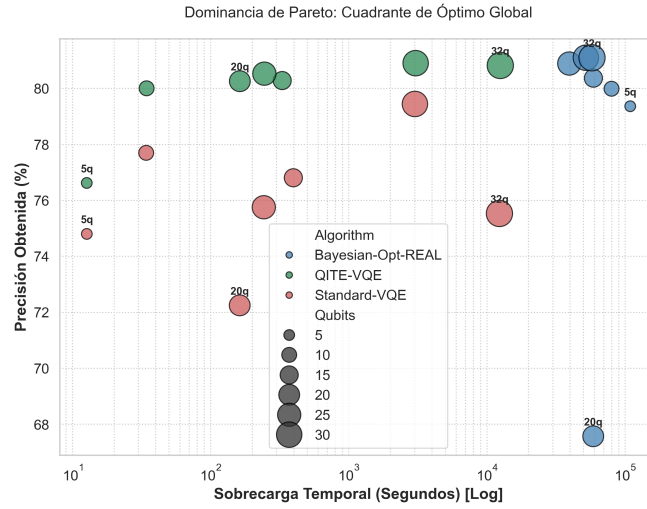


Figura 5.5: Mapa de eficiencia de Pareto: Compromiso entre precisión alcanzada y tiempo consumido.

Para ver de manera más detallada los tiempos de los algoritmos, como se puede observar, QITE-VQE y VQE presentan unos tiempos prácticamente idénticos a pesar de tener precisiones con gran diferencia. A medida que aumenta el número de combinaciones posibles, es decir, el número de qubits, aumenta de manera proporcional el tiempo de ejecución en los algoritmos cuánticos. En cambio en optimización bayesiana se mantiene estable, subiendo y bajando muy ligeramente, en cada cambio de escala. Estos resultados temporales se explican ya que mientras que el optimizador clásico solo ve una arquitectura por iteración, el algoritmo cuántico, para poder operar, tiene que 'sostener' en su superposición matemática la totalidad del espacio de Hilbert, cuya dimensión se duplica con cada qubit añadido, forzando al hardware de simulación clásico a realizar un esfuerzo exponencialmente mayor.

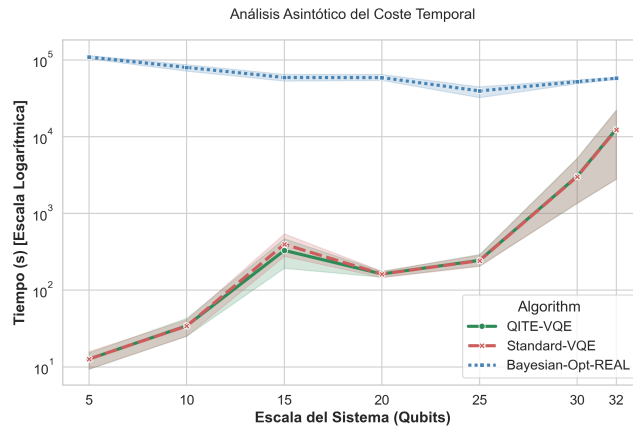


Figura 5.6: Crecimiento logarítmico del tiempo de simulación.

En resumen del apartado temporal:

- **En computación cuántica (simulada clásicamente):** El tiempo de ejecución crece de forma exponencial debido a que el simulador procesa matrices que duplican su tamaño por cada qubit añadido ($2^5 \rightarrow 2^{10} \rightarrow \dots \rightarrow 2^{32}$).
- **En Optimización Bayesiana:** El tiempo de ejecución se mantiene lineal y es independiente de la escala. Esto se debe a que el algoritmo evalúa únicamente 150 combinaciones específicas por experimento, independientemente del tamaño total del espacio de búsqueda.

Capítulo 6

Discusión de los Resultados

En este capítulo se analizan de manera crítica los resultados empíricos expuestos en el capítulo anterior, verificando las hipótesis de partida y contrastándolas con el estado del arte detallado en el Capítulo 2. Asimismo, se evalúan las implicaciones transversales que la adopción de la Inteligencia Artificial Cuántica (QML) conlleva a nivel socioeconómico, ético y legal.

6.1. Verificación de las Hipótesis y Cierre de la Brecha Estructural

En la sección 2.2.2 se planteó que este trabajo buscaba llenar un hueco fundamental en la literatura: no limitarse a comparar precisiones finales, sino evaluar la viabilidad estructural real de los algoritmos en la era NISQ. Los resultados obtenidos desde los 5 hasta los 32 qubits han verificado esta hipótesis de viabilidad con éxito.

Tal y como se teorizó mediante la formulación del operador de tiempo imaginario ($e^{-\tau\hat{H}}$), el algoritmo ha demostrado actuar como un filtro cuántico altamente eficiente. Las gráficas de escalabilidad del Capítulo 5 confirman empíricamente que este filtro "suprime" de manera natural las configuraciones de baja precisión sin necesidad de construir circuitos de gran profundidad. Al reportar analíticamente el número de puertas CNOT, se evidencia que QITE-VQE logra mantener un $\approx 80\%$ de precisión sosteniendo circuitos mucho más superficiales que los enfoques cuánticos puros tradicionales.

Por otro lado, ha quedado demostrado que además de ser robusto al mantener precisión estable (a pesar del ruido, escala y semilla), también es un éxito en cuanto a tiempos de ejecución, ya que se consiguen tiempos mucho menores con ambos algoritmos cuánticos implementados. Esto se debe gracias a justamente las ventajas comentadas sobre la computación cuántica en la sección 2.2. Esta

ventaja cabe destacar que es bajo simulación en un hardware clásico, por lo que sobre un hardware cuántico debería ser aún incluso mayor.

6.2. Comparativa con los Baselines del Estudio

El análisis comparativo frente a las dos estrategias algorítmicas evaluadas en este trabajo (Standard-VQE y Optimización Bayesiana) revela mejoras empíricas sustanciales en la estabilidad y robustez del proceso de optimización.

6.2.1. Frente al Baseline Cuántico: Standard-VQE

La comparativa directa entre ambas aproximaciones cuánticas variacionales demuestra la efectividad del filtrado por tiempo imaginario. Mientras que el Standard-VQE requiere optimizar heurísticamente sobre un paisaje energético complejo y a menudo no convexo, QITE-VQE aplica el operador $e^{-\tau\hat{H}}$ guiando al sistema de forma determinista hacia el estado fundamental.

Desde el punto de vista estructural, esta optimización se traduce en una menor demanda de profundidad y compuertas CNOT en los experimentos de escalabilidad. Esta reducción de recursos cuánticos es la responsable directa de que QITE-VQE haya tolerado con éxito tasas de ruido de hasta el 3 % por compuerta (un entorno hostil que castiga fuertemente el entrelazamiento, fallando un 30 % de las veces cada puerta), mientras que el Standard-VQE proyecta bajadas bruscas y colapsos de fidelidad bajo las mismas condiciones NISQ debido a la propagación acumulada de errores de puerta. Esto se ve sobre todo en la escala de 20 qubits, en donde la semilla 42 provoca una caída muy pronunciada en VQE estándar mientras que QITE-VQE se mantiene estable.

En cuanto a tiempo de cómputo ambos algoritmos son prácticamente iguales, y van aumentando de manera exponencial junto con cada aumento de escalas, por lo que no se aporta ninguna ventaja temporal frente al baseline cuántico.

6.2.2. Frente al Baseline Clásico: Optimización Bayesiana

Los resultados de la comparativa clásica contra la Optimización Bayesiana clásica (basada en procesos gaussianos subrogados) confirman el valor añadido de la superposición cuántica para la exploración de arquitecturas.

La HPO bayesiana tradicional, aunque es eficiente en términos de tiempo secuencial clásico, demostró una dependencia crítica de la inicialización aleatoria. El colapso estadístico observado en la "Semilla 42" para la escala de 20 qubits es un claro indicador de cómo las aproximaciones clásicas pueden quedar atrapadas

indefinidamente en mínimos locales profundos del paisaje de hiperparámetros. En contraste, QITE-VQE ofrece una distribución de resultados extremadamente compacta y predecible, aún incluso cuando se aumenta el ruido hasta 0.03 para la simulación cuántica. Al mapear el espacio global a través del operador hamiltoniano, el algoritmo cuántico logra esquivar dichos colapsos catastróficos, garantizando la convergencia al óptimo con una varianza mínima y situándose de forma dominante en la frontera de Pareto Eficiente.

Por otro lado, en esta comparación si se presentan claras diferencias temporales. Se presenta un menor coste temporal por ambos algoritmos cuánticos frente a Optimización bayesiana. Como se puede observar a favor de esta última es que el tiempo se mantiene estable, incluso casi llegando a bajar conforme aumenta la escala, mientras tanto en cuántica, la escala temporal aumenta casi de manera exponencial, dejando claro que si se llega a aumentar la escala por encima de 32 qubits, probablemente llegue a ser más lenta la simulación cuántica. Merece la pena recordar que todas estas comparaciones son simulaciones bajo hardware clásico (FinisTerra III), por lo que bajo un ordenador cuántico real se podría exprimir aún más las ventajas de superposición, entrelazamiento entre otras, realizando las ejecuciones en menor tiempo probablemente, como se espera y expone en la literatura científica.

6.3. Implicaciones Transversales

La adopción de arquitecturas híbridas cuántico-clásicas para la optimización de hiperparámetros (HPO) trasciende el ámbito puramente técnico de la ingeniería del software. El diseño automatizado de modelos complejos como los Transformers mediante algoritmos cuánticos genera un impacto directo y medible en dimensiones críticas de la sociedad tecnológica actual.

6.3.1. Aspectos Éticos y Medioambientales: Hacia una IA Verde

Actualmente, el ecosistema de la Inteligencia Artificial se enfrenta a una crisis de sostenibilidad medioambiental. El entrenamiento de grandes modelos de lenguaje (LLMs) y arquitecturas generativas profundas requiere clústeres masivos de GPUs funcionando ininterrumpidamente durante meses, lo que genera una huella de carbono exorbitante. Esta problemática se agrava exponencialmente durante la fase de Optimización de Hiperparámetros, donde enfoques tradicionales como *Grid Search* o búsquedas estocásticas obligan a entrenar miles de modelos subóptimos desde cero únicamente para descartarlos.

La integración de co-procesadores cuánticos para delegar esta carga compu-

tacional representa un paso estratégico hacia el paradigma de la 'IA Verde' (*Green AI*). Algoritmos como QITE-VQE, al explotar el paralelismo cuántico y el filtrado por tiempo imaginario, tienen el potencial de suprimir drásticamente el número de inferencias clásicas necesarias para aislar la topología óptima de una red neuronal. Desde un punto de vista ético, la transición hacia la IA Cuántica no solo acelera la innovación tecnológica, sino que provee una solución técnica al imperativo moral de mitigar el impacto climático del desarrollo algorítmico, desacoplando el avance de la Inteligencia Artificial del consumo desmedido de recursos energéticos.

6.3.2. Aspectos Socioeconómicos: Democratización y Competitividad

El desarrollo de la Inteligencia Artificial contemporánea tiende hacia un oligopolio dominado por corporaciones tecnológicas internacionales, las únicas con el músculo financiero suficiente para asumir los costes computacionales de la búsqueda por fuerza bruta. En este contexto socioeconómico, la optimización cuántica eficiente actúa como un factor disruptivo.

Al reducir el tiempo y el coste del ciclo de diseño, la Inteligencia Artificial Cuántica (QML) habilita a universidades, centros de investigación públicos y pequeñas o medianas empresas (pymes) a entrenar modelos fundacionales altamente precisos sin depender de infraestructuras corporativas prohibitivas. A nivel regional y estatal, la capacidad demostrada en este trabajo para operar sobre simuladores de alta escala subraya el valor estratégico de las inversiones públicas en infraestructuras avanzadas. Proyectos pioneros como el hardware cuántico Qmio alojado en el Supercomputador FinisTerra III del CESGA no solo proporcionan independencia tecnológica, sino que posicionan al ecosistema académico y empresarial local a la vanguardia de un sector que redefinirá el tejido productivo global, impulsando la retención de talento y la creación de empleo de alto valor añadido.

6.3.3. Aspectos Legales y Culturales: El Nuevo Paradigma del QML

El diseño automatizado de redes neuronales impulsado por leyes de la mecánica cuántica abre debates inéditos en el panorama legal y regulatorio. Actualmente, las leyes de propiedad intelectual y patentes se enfrentan a lagunas considerables respecto a las invenciones autogeneradas por algoritmos clásicos. La irrupción de algoritmos de optimización basados en superposición probabilística añade una nueva capa de complejidad: surge el reto legal de determinar la autoría, trazabilidad y los derechos de explotación sobre arquitecturas de software cuyas

configuraciones óptimas han sido decididas por el colapso de un estado cuántico. Este trabajo evidencia la necesidad urgente de marcos regulatorios actualizados que contemplen la hibridación cuántico-clásica en la IA.

Finalmente, desde una perspectiva cultural y educativa, este proyecto contribuye a la necesaria desmitificación de la computación cuántica. Al sacar esta tecnología del terreno puramente teórico y de la física fundamental para aplicarla a problemas reales y contemporáneos de MLOps (*Machine Learning Operations*), se evidencia un cambio de paradigma. Este avance subraya la necesidad de adaptar la formación académica en inteligencia artificial, preparando a las futuras generaciones de desarrolladores para transicionar desde la programación imperativa clásica hacia la algoritmia cuántica aplicada.

Capítulo 7

Conclusiones y posibles ampliaciones

Este capítulo final sintetiza las aportaciones empíricas y teóricas desarrolladas a lo largo del presente Trabajo de Fin de Grado. Asimismo, se exponen de manera transparente las limitaciones inherentes a la simulación en la actual era cuántica y se proponen diversas líneas de investigación para dar continuidad tecnológica y académica al proyecto.

7.1. Resumen de las Principales Aportaciones

El objetivo central de este proyecto era investigar y validar la eficacia de la Inteligencia Artificial Cuántica —específicamente los algoritmos variacionales filtrados— para resolver problemas intratables de Optimización de Hiperparámetros (HPO) en modelos profundos como los Transformers. Tras el diseño de la arquitectura y la ejecución de la masiva campaña experimental en el Supercomputador FinisTerra III y el ordenador cuántico Qmio, las aportaciones principales se resumen en:

- **Implementación Cuántico-Clásica Funcional:** Se ha desarrollado con éxito un pipeline completo (QITE-VQE y VQE) capaz de codificar el espacio de hiperparámetros de un modelo Transformer en un Hamiltoniano, operando como enlace directo entre arquitecturas de Deep Learning modernas y computación cuántica.
- **Estudio de Escalabilidad Masiva (5 a 32 Qubits):** A diferencia de la literatura convencional, restringida a pruebas de concepto pequeñas, este trabajo ha aportado una auditoría estructural sistemática hasta los 32 qubits (espacios de búsqueda masivos, de 2^{32} combinaciones), documentando métricas como precisión, tiempo, puertas CNOT y profundidad.

- **Demostración Empírica de Tolerancia al Ruido (NISQ):** Se ha probado empíricamente que la reducción de profundidad de circuito lograda por la evolución de tiempo imaginario otorga a QITE-VQE la capacidad de mantener una precisión de $\approx 80\%$ bajo modelos de despolarización severos (hasta un 30% de error en compuertas entrelazadas), umbral que destruye por completo los enfoques tradicionales como se vio con un VQE estándar.
- **Inmunidad a Mínimos Locales (Barren Plateaus):** El análisis estadístico evidenció, mediante la detección de anomalías (ej. Semilla 42), que tanto la optimización bayesiana clásica como el VQE Estándar son susceptibles a colapsos catastróficos por inicialización. El algoritmo propuesto esquivó con éxito estas trampas, validando que el filtrado temporal amplifica el estado fundamental de forma global y determinista.
- **Mapeo de Eficiencia de Pareto:** Se ha construido una demostración gráfica posicionando a QITE-VQE en la frontera óptima de compromiso entre la sobrecarga de tiempo asintótico y la precisión final alcanzada frente a búsquedas estocásticas y de fuerza bruta.

7.2. Suposiciones y Limitaciones del Estudio

A pesar de la robustez de los resultados obtenidos y de la validación satisfactoria en hardware real, el estado actual de la tecnología cuántica impone ciertas limitaciones técnicas que definen el alcance de las conclusiones:

- **Fidelidad y Conectividad en Hardware Real (Qmio):** Aunque el algoritmo ha sido validado en la QPU superconductora de 32 qubits del CESGA, la ejecución física se encuentra sujeta a la topología específica del procesador. A diferencia de los entornos simulados ideales, el ruido en hardware real no es puramente estocástico; fenómenos como la diafonía (*crosstalk*) y las fluctuaciones temporales en los tiempos de relajación T_1 y T_2 pueden introducir sesgos en la convergencia del oráculo que no son totalmente capturados en modelos de ruido estáticos.
- **Especificidad de la Arquitectura:** Los resultados de resiliencia observados están estrechamente vinculados a procesadores de tipo *transmon*. El modelo de error despolarizante utilizado en la fase de simulación se calibró para imitar la asimetría de puertas de Qmio (donde las operaciones CNOT son un orden de magnitud más ruidosas que las rotaciones locales). No obstante, otras arquitecturas como las trampas de iones o los átomos neutros presentan perfiles de error y conectividades *all-to-all* que podrían alterar la eficiencia relativa de QITE-VQE frente a otros métodos.

- **Escalabilidad de la Optimización Clásica:** La Optimización Bayesiana utilizada como *baseline* ha demostrado ser competitiva en escalas bajas, pero el coste de actualizar el modelo subrogado tras cada uno de los 5250 entrenamientos totales realizados en el estudio comienza a ser un factor crítico al alcanzar los 32 qubits. La viabilidad a largo plazo depende de la capacidad de los optimizadores clásicos para gestionar espacios de Hilbert de tal dimensionalidad sin incurrir en una latencia inasumible.

7.3. Vías de Mejora y Trabajo Futuro

La superación de la fase de simulación y el despliegue exitoso en hardware físico abren nuevas vías de investigación para expandir el potencial de este trabajo:

- **Implementación de Mitigación de Errores (Error Mitigation):** Tras observar el comportamiento del algoritmo en Qmio, se propone como paso inmediato la integración de técnicas como la Extrapolación de Cero Ruido (ZNE) o el *Probabilistic Error Cancellation* (PEC). Estas estrategias permitirían 'limpiar' los valores esperados de los observables obtenidos en la QPU, acercando los resultados experimentales a los límites teóricos ideales.
- **Exploración de Ansatz Dinámicos (ADAPT-VQE):** Con el fin de optimizar aún más los recursos cuánticos, se plantea reemplazar la estructura estática de *EfficientSU2* por algoritmos dinámicos. Esto permitiría construir circuitos que crezcan únicamente en las direcciones del espacio de parámetros que maximicen el gradiente de precisión, minimizando el número de puertas CNOT y reduciendo la exposición a la decoherencia térmica.
- **Optimización Multiobjetivo y Compresión Cuántica:** Se propone extender el Hamiltoniano de penalización actual para que no solo busque la máxima precisión del modelo Transformer, sino que considere simultáneamente la reducción de la huella computacional (poda de la red) y la eficiencia energética de la inferencia. Esto posicionaría a QITE-VQE como una herramienta clave para el diseño de modelos de lenguaje más ligeros y sostenibles.
- **Benchmarking en Arquitecturas Alternativas:** Para validar la generalización de la resiliencia algorítmica, sería de gran interés ejecutar el flujo de trabajo en procesadores de naturaleza no superconductora. Comparar el desempeño de los filtros de tiempo imaginario en diferentes dispositivos NISQ permitiría establecer un estándar de viabilidad universal para la optimización de inteligencia artificial a escala industrial.

Bibliografía

- [1] A. J, A. Adedoyin, J. Ambrosiano et al., «Quantum algorithm implementations for beginners,» *ACM Transactions on Quantum Computing*, vol. 3, n.º 4, págs. 1-92, 31 de dic. de 2022.
- [2] S. Rashid, «Understanding quantum computing and its policy implications,» mar. de 2026.
- [3] C. Hughes, J. Isaacson, A. Perry, R. F. Sun y J. Turner, «What is a qubit?» En *Quantum Computing for the Quantum Curious*. Cham: Springer International Publishing, 2021, págs. 7-16.
- [4] I. Y. Doskoch y M. A. Man'ko, «Superposition principle and born's rule in the probability representation of quantum states,» *Quantum Reports*, vol. 1, n.º 2, págs. 130-150, 26 de sep. de 2019.
- [5] A. A. Gorbatshevich y N. M. Shubin, «Quantum logic gates,»
- [6] M. Bataille, *Quantum circuits of CNOT gates*, 16 de dic. de 2020.
- [7] S. Nishio, Y. Pan, T. Satoh, H. Amano y R. V. Meter, «Extracting success from IBM's 20-qubit machines using error-aware compilation,» *ACM Journal on Emerging Technologies in Computing Systems*, vol. 16, n.º 3, págs. 1-25, 31 de jul. de 2020.
- [8] S. Zhang y L. Li, «A brief introduction to quantum algorithms,» *CCF Transactions on High Performance Computing*, vol. 4, n.º 1, págs. 53-62, mar. de 2022.
- [9] «Quantum-inspired Optimization Algorithms for Scalable Machine Learning Models,» en, *International Journal of Intelligent Engineering and Systems*, vol. 18, n.º 10, págs. 275-290, nov. de 2025.
- [10] J. Jarsania y C. Chavda, «Quantum-Inspired Algorithms for Large-Scale Data Analytics: A Tensor Network Approach,» en, en *2026 IEEE 16th Annual Computing and Communication Workshop and Conference (CCWC)*, Las Vegas, NV, USA: IEEE, ene. de 2026, págs. 0317-0323.
- [11] M. Shan, Y. Halimu, J. Sun, W. Fang y V. Palade, «A swarm intelligence optimization algorithm on riemannian manifolds,» en, *Swarm and Evolutionary Computation*, vol. 101, pág. 102319, feb. de 2026.

- [12] Y. Li, J. Guo, C. Zhang, X. Wang y L. Guo, «Hybrid intelligent model for strength prediction and parameter sensitivity analysis of cemented paste backfill,» en, *MetaResource*, págs. 1-21, 2026.
- [13] Q. Qiu, M. Wu, Q. Sun, X. Li y H. Xu, *A Novel Quantum Algorithm for Ant Colony Optimization*, en, arXiv:2403.00367 [quant-ph], mar. de 2024.
- [14] M. G. De Andoin y J. Echanobe, «Implementable hybrid quantum ant colony optimization algorithm,» en, *Quantum Machine Intelligence*, vol. 4, n.º 2, pág. 12, dic. de 2022.
- [15] P. A. Viana y F. M. d. P. Neto, *A Quantum Genetic Algorithm Framework for the MaxCut Problem*, en, arXiv:2501.01058 [quant-ph], ene. de 2025.
- [16] P. Sridhar y H. M. Kittur, «Analog Integrated Circuit Optimization With an Enhanced Adaptive Quantum Genetic Algorithm,» en, *IEEE Access*, vol. 14, págs. 19 205-19 223, 2026.
- [17] N. Xie, X. Lee, T. Chen, Y. Saito, N. Asai y D. Cail, «An Adaptive Weighted Qite-Vqe Algorithm for Combinatorial Optimization Problems,» en, en *2025 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Albuquerque, NM, USA: IEEE, ago. de 2025, págs. 208-218.
- [18] X. Jiang, Z. Sheng, C. Gong, W. Li y Z. Shuai, «Measurement-efficient ADAPT-VQE with the SOAP parameter optimizer,» *The Journal of Chemical Physics*, vol. 163, n.º 22, pág. 224 118, 14 de dic. de 2025.
- [19] J. Obst, F. Truger, J. Barzen y F. Leymann, «Parameter Transfers for Warm-Started QAOA:» en, en *Proceedings of the 1st International Conference on Quantum Software*, Bilbao, Spain: SCITEPRESS - Science y Technology Publications, 2025, págs. 37-48.
- [20] A. Lusso, C. N. Gimenez y A. Mata Ali, «Benchmarking QAOA on the Job Reassignment Problem: An Empirical Analysis using Transfer Learning and TQA Initialisation,» en, *SADIO Electronic Journal of Informatics and Operations Research*, vol. 25, n.º 1, e090, mar. de 2026.
- [21] E. Bae y S. Lee, «Recursive QAOA outperforms the original QAOA for the MAX-CUT problem on complete graphs,» en, *Quantum Information Processing*, vol. 23, n.º 3, pág. 78, feb. de 2024.
- [22] E. Jang, Z. Chen, D. Ha et al., «A Cyclic Layerwise QAOA Training,» en, *Quantum Machine Intelligence*, vol. 8, n.º 1, pág. 5, jun. de 2026, arXiv:2601.20029 [quant-ph].
- [23] G. Miki e Y. Tokura, *A New Scaling Function for QAOA Tensor Network Simulations*, en, arXiv:2505.23256 [quant-ph], mayo de 2025.
- [24] L. A. M. Rattighieri, P. M. Prado, M. C. d. Oliveira y F. F. Fanchini, *Measurement-Guided State Refinement for Shallow Feedback-Based Quantum Optimization Algorithm*, en, arXiv:2602.20407 [quant-ph], feb. de 2026.

- [25] M. Legnini y J. Berberich, «Robust Feedback-Based Quantum Optimization: Analysis of Coherent Control Errors,» en, en *2025 IEEE International Conference on Quantum Control, Computing and Learning (qCCL)*, Hong Kong, Hong Kong: IEEE, jun. de 2025, págs. 17-22.
- [26] J. V. Pankkonen, M. Raasakka, A. Marchesin e I. Tittonen, *Enhancing Hybrid Methods in Parameterized Quantum Circuit Optimization*, en, arXiv:2510.08142 [quant-ph], oct. de 2025.
- [27] L. Arceci, V. Kuzmin y R. V. Bijnen, *Gaussian process model kernels for noisy optimization in variational quantum algorithms*, en, arXiv:2412.13271 [quant-ph], dic. de 2024.
- [28] J. Pankkonen, L. Ylinen, M. Raasakka, A. Marchesin e I. Tittonen, *Gate Freezing Method for Gradient-Free Variational Quantum Algorithms in Circuit Optimization*, en, arXiv:2507.07742 [quant-ph], jul. de 2025.
- [29] Y. Sato, H. C. Watanabe, R. Raymond et al., «Variational quantum algorithm for generalized eigenvalue problems and its application to the finite-element method,» en, *Physical Review A*, vol. 108, n.º 2, pág. 022429, ago. de 2023.
- [30] I. Sajjad, H. M. Alshanbari, M. M. A. Almazah et al., «Adaptive Grover-driven optimization for quantum-inspired deep learning: A gradient-free training framework,» en, *AIMS Mathematics*, vol. 10, n.º 11, págs. 26568-26592, 2025.
- [31] H. Ominato, T. Ohyama y K. Yamaguchi, «Grover Adaptive Search With Fewer Queries,» en, *IEEE Access*, vol. 12, págs. 74619-74632, 2024.
- [32] N. H. Viet, N. X. Tung, T. Van Chien y W.-J. Hwang, «Advanced Quantum Annealing for the Bi-Objective Traveling Thief Problem: An ϵ -Constraint-Based Approach,» en, *IEEE Transactions on Quantum Engineering*, págs. 1-13, 2026.
- [33] N. Kowshik, S. Manna y S. P. Pal, *Quantum Approximate and Quantum Walk Optimization Approaches to Set Balancing*, en, arXiv:2509.07200 [quant-ph], sep. de 2025.
- [34] M. Zering, J. Joyce, T. Gurfinkel y J. Wang, *Benchmarking Lie-Algebraic Pretraining and Non-Variational QWOA for the MaxCut Problem*, en, arXiv:2512.22856 [quant-ph], dic. de 2025.
- [35] Y. Sun, J. Liu, Z. Wu, V. Tresp e Y. Ma, *SA-DQAS: Self-attention Enhanced Differentiable Quantum Architecture Search*, en, arXiv:2406.08882 [quant-ph], ago. de 2025.
- [36] S.-X. Zhang, C.-Y. Hsieh, S. Zhang y H. Yao, «Differentiable Quantum Architecture Search,» en, *Quantum Science and Technology*, vol. 7, n.º 4, pág. 045023, oct. de 2022, arXiv:2010.08561 [quant-ph].

- [37] Y. Sun, J. Liu, Y. Ma y V. Tresp, «Differentiable Quantum Architecture Search For Job Shop Scheduling Problem,» en, en *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Seoul, Korea, Republic of: IEEE, abr. de 2024, págs. 236-240.
- [38] Y. Li, R. Liu, X. Hao, R. Shang, P. Zhao y L. Jiao, «EQNAS: Evolutionary Quantum Neural Architecture Search for Image Classification,» en, *Neural Networks*, vol. 168, págs. 471-483, nov. de 2023.
- [39] S. B. Son y S. Park, «Q-RLONAS: Towards Efficient Quantum Neural Architecture Search,» en, en *2025 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Albuquerque, NM, USA: IEEE, ago. de 2025, págs. 1756-1765.
- [40] C. Chowdhury, «Exploring Quantum Support Vector Regression for Predicting Hydrogen Storage Capacity of Nanoporous Materials,» en, *Advanced Intelligent Discovery*, e202500015, nov. de 2025.
- [41] R. Ahmad, M. Kashif, N. Innan y M. Shafique, *Quantum vs. Classical Machine Learning: A Benchmark Study for Financial Prediction*, en, arXiv:2601.03802 [cs], ene. de 2026.
- [42] R. Coelho, A. Sequeira y L. Paulo Santos, «VQC-based reinforcement learning with data re-uploading: performance and trainability,» en, *Quantum Machine Intelligence*, vol. 6, n.º 2, pág. 53, dic. de 2024.
- [43] T. Yu y H. Zhu, *Hyper-parameter optimization: A review of algorithms and applications*, 12 de mar. de 2020.
- [44] J.-S. Hwang, S.-S. Lee, J.-W. Gil y C.-K. Lee, «Determination of optimal batch size of deep learning models with time series data,» *Sustainability*, vol. 16, n.º 14, pág. 5936, 12 de jul. de 2024.
- [45] P. Liashchynskiy y P. Liashchynskiy, «Grid search, random search, genetic algorithm: A big comparison for NAS,»
- [46] X. Wang, Y. Jin, S. Schmitt y M. Olhofer, «Recent advances in bayesian optimization,» *ACM Computing Surveys*, vol. 55, n.º 13, págs. 1-36, 31 de dic. de 2023.
- [47] F. F. Flöther, «The state of quantum computing applications in health and medicine,» *Research Directions: Quantum Technologies*, págs. 1-21, 24 de jul. de 2023.
- [48] A. Hassan, «Quantum-IoT security: Advances, challenges, and future directions,» *Journal of Hardware and Systems Security*, vol. 10, n.º 1, pág. 6, dic. de 2026.
- [49] J. Singh y K. S. Bhangu, «Contemporary quantum computing use cases: Taxonomy, review and challenges,» *Archives of Computational Methods in Engineering*, vol. 30, n.º 1, págs. 615-638, ene. de 2023.

- [50] A. A. Meléndez, C. G. Almudéver, M. A. Garcia-March et al., *Adaptive time compressed QITE (ACQ) and its geometrical interpretation*, Version Number: 1, 2025.

Apéndice A

Manuales técnicos

El código fuente íntegro desarrollado para este trabajo, así como los conjuntos de datos experimentales y los **scripts** de supercomputación, se encuentran disponibles de forma pública en el repositorio de GitHub: https://github.com/fedellor/TFG_QITE_HPO_Reproducibility.

Este repositorio ha sido diseñado bajo un paradigma de reproducibilidad estricta, conteniendo exclusivamente los archivos necesarios para ejecutar el **pipeline** algorítmico final y generar los resultados empíricos expuestos a lo largo de este documento.

A.1. Estructura de ficheros

A continuación, se detalla la topología del repositorio, agrupada en módulos funcionales para facilitar la trazabilidad experimental:

- **src/**: Directorio que conforma el núcleo algorítmico del estudio. Contiene el código fuente maestro de la resolución de hiperparámetros.
 - **benchmark_bayesiano_real.py**: El orquestador de nivel clásico. Aplica el algoritmo clásico sobre el espacio de hiperparámetros y sirve como punto de entrada principal.
 - **qite_vqe_gpu.py**: Implementación nativa de la rutina cuántica QITE-VQE y VQE estándar. Instancia el *ansatz* paramétrico e inyecta dinámicamente los tensores de ruido.
 - **benchmark_subrogado.py**: Evaluador clásico puente que invoca la arquitectura de la red neuronal durante el ciclo de optimización iterativo.
 - **hpo_spaces.py**: Mapa de discretización que traduce los estados de superposición cuántica a hiperparámetros continuos.

- `transformer_model/`: Topología íntegra de la red neuronal subrogada implementada en PyTorch.
- `data_processors/`: Tuberías clásicas encargadas de preprocesar los registros de eventos de los que se alimenta la red.
- `slurm/`: Entorno de orquestación HPC. Colección de scripts *bash* configurados para el envío de trabajos de cálculo masivo al clúster de supercomputación FinisTerae III (CESGA).
 - `job_bayesiano_real.sh`: Script maestro para ejecuciones unitarias.
 - `job_25q.sh`, `job_32q.sh`, etc.: Enrutadores dedicados para inyectar parametrizaciones específicas de memoria RAM según la dimensionalidad del espacio de Hilbert evaluado.
- `data/`: Datos en bruto y modelos preentrenados.
 - Contiene los archivos de eventos particionados y los pesos neuronales estáticos del evaluador, aislados para evitar dependencias externas durante la ejecución en los nodos de cómputo.
- `analysis/`: Análisis estadístico y generación visual.
 - `merge_final_operativo.py`: Motor de agregación que consolida los archivos CSV fragmentados procedentes de los trabajos del clúster en una matriz única.
 - `generar_graficas_TFG.py` y `plot_structural.py`: Rutinas de renderizado estadístico en alta resolución para las figuras presentadas en esta memoria.
- `results/`: Registro telemetrado.
 - `resultados_MAESTRO_TFG.csv`: Base de datos experimental exhaustiva resultante de consolidar las iteraciones del clúster.

A.2. Creación del entorno

Para asegurar la estabilidad en la reproducción de los cálculos y evitar conflictos de dependencias, se recomienda utilizar un sistema operativo basado en Linux y gestionar el entorno virtual mediante `conda` o `venv`.

Las instrucciones precisas para inicializar el entorno desde una terminal son las siguientes:

1. Clonar el repositorio en el sistema local o en el nodo de acceso del clúster HPC:

```
git clone https://github.com/fedellor/TFG_QITE_HPO_Reproducibility.git
cd TFG_QITE_HPO_Reproducibility
```

2. Crear un entorno virtual con Python 3.10 o superior y activarlo:

```
python3 -m venv entornoTFG
source entornoTFG/bin/activate
```

3. Instalar los paquetes fundamentales (tales como `qiskit`, `torch`, y `scikit-optimize`) fijados estrictamente en el árbol de dependencias:

```
pip install -r requirements.txt
```

A.3. Reproducción de los resultados

Debido a la magnitud del espacio de Hilbert implicado (simulaciones de hasta 32 *qubits* con inyección de ruido de despolarización), es **imprescindible** disponer de infraestructura HPC. El volumen computacional de estos cálculos hace inviable su ejecución en ordenadores personales.

Para repetir íntegramente la experimentación empírica expuesta en el Capítulo 6, siga los siguientes pasos:

1. **Ejecución algorítmica (HPC):** Desde el nodo de acceso del supercomputador CESGA, envíe el trabajo a la cola de particiones de cálculo mediante el gestor Slurm:

```
sbatch slurm/job_bayesiano_real.sh slurm/lanzar_todo.sh
```

Este proceso distribuirá automáticamente las 335 iteraciones paramétricas (combinando escalas, semillas y niveles de ruido) a través de los múltiples núcleos físicos y la memoria RAM solicitada tanto en clásica (Optimización bayesiana), como en cuántica.

2. **Consolidación de Telemetría:** Una vez finalizados los trabajos paralelos en el clúster, extraiga los datos en bruto y consolídelos en el archivo maestro ejecutando:

```
python analysis/merge_final_operativo.py
```

3. **Mapeo Visual:** Para compilar matemáticamente la huella cuántica estructural (*Depth* y CNOTs) y renderizar las curvas termodinámicas y diagramas de Pareto resultantes, invoque el motor analítico:

```
python analysis/generar_graficas_TFG.py  
python analysis/plot_structural.py
```

Apéndice B

Licencia

Apache License
Version 2.0, January 2004
<http://www.apache.org/licenses/>

TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

1. Definitions.

"License" shall mean the terms and conditions for use, reproduction, and distribution as defined by Sections 1 through 9 of this document.

"Licensor" shall mean the copyright owner or entity authorized by the copyright owner that is granting the License.

"Legal Entity" shall mean the union of the acting entity and all other entities that control, are controlled by, or are under common control with that entity. For the purposes of this definition, "control" means (i) the power, direct or indirect, to cause the direction or management of such entity, whether by contract or otherwise, or (ii) ownership of fifty percent (50%) or more of the outstanding shares, or (iii) beneficial ownership of such entity.

"You" (or "Your") shall mean an individual or Legal Entity exercising permissions granted by this License.

"Source" form shall mean the preferred form for making modifications, including but not limited to software source code, documentation source, and configuration files.

"Object" form shall mean any form resulting from mechanical

transformation or translation of a Source form, including but not limited to compiled object code, generated documentation, and conversions to other media types.

"Work" shall mean the work of authorship, whether in Source or Object form, made available under the License, as indicated by a copyright notice that is included in or attached to the work (an example is provided in the Appendix below).

"Derivative Works" shall mean any work, whether in Source or Object form, that is based on (or derived from) the Work and for which the editorial revisions, annotations, elaborations, or other modifications represent, as a whole, an original work of authorship. For the purposes of this License, Derivative Works shall not include works that remain separable from, or merely link (or bind by name) to the interfaces of, the Work and Derivative Works thereof.

"Contribution" shall mean any work of authorship, including the original version of the Work and any modifications or additions to that Work or Derivative Works thereof, that is intentionally submitted to Licensor for inclusion in the Work by the copyright owner or by an individual or Legal Entity authorized to submit on behalf of the copyright owner. For the purposes of this definition, "submitted" means any form of electronic, verbal, or written communication sent to the Licensor or its representatives, including but not limited to communication on electronic mailing lists, source code control systems, and issue tracking systems that are managed by, or on behalf of, the Licensor for the purpose of discussing and improving the Work, but excluding communication that is conspicuously marked or otherwise designated in writing by the copyright owner as "Not a Contribution."

"Contributor" shall mean Licensor and any individual or Legal Entity on behalf of whom a Contribution has been received by Licensor and subsequently incorporated within the Work.

2. Grant of Copyright License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable copyright license to reproduce, prepare Derivative Works of, publicly display, publicly perform, sublicense, and distribute the Work and such Derivative Works in Source or Object form.

3. Grant of Patent License. Subject to the terms and conditions of

this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable (except as stated in this section) patent license to make, have made, use, offer to sell, sell, import, and otherwise transfer the Work, where such license applies only to those patent claims licensable by such Contributor that are necessarily infringed by their Contribution(s) alone or by combination of their Contribution(s) with the Work to which such Contribution(s) was submitted. If You institute patent litigation against any entity (including a cross-claim or counterclaim in a lawsuit) alleging that the Work or a Contribution incorporated within the Work constitutes direct or contributory patent infringement, then any patent licenses granted to You under this License for that Work shall terminate as of the date such litigation is filed.

4. Redistribution. You may reproduce and distribute copies of the Work or Derivative Works thereof in any medium, with or without modifications, and in Source or Object form, provided that You meet the following conditions:
 - (a) You must give any other recipients of the Work or Derivative Works a copy of this License; and
 - (b) You must cause any modified files to carry prominent notices stating that You changed the files; and
 - (c) You must retain, in the Source form of any Derivative Works that You distribute, all copyright, patent, trademark, and attribution notices from the Source form of the Work, excluding those notices that do not pertain to any part of the Derivative Works; and
 - (d) If the Work includes a "NOTICE" text file as part of its distribution, then any Derivative Works that You distribute must include a readable copy of the attribution notices contained within such NOTICE file, excluding those notices that do not pertain to any part of the Derivative Works, in at least one of the following places: within a NOTICE text file distributed as part of the Derivative Works; within the Source form or documentation, if provided along with the Derivative Works; or, within a display generated by the Derivative Works, if and wherever such third-party notices normally appear. The contents of the NOTICE file are for informational purposes only and

do not modify the License. You may add Your own attribution notices within Derivative Works that You distribute, alongside or as an addendum to the NOTICE text from the Work, provided that such additional attribution notices cannot be construed as modifying the License.

You may add Your own copyright statement to Your modifications and may provide additional or different license terms and conditions for use, reproduction, or distribution of Your modifications, or for any such Derivative Works as a whole, provided Your use, reproduction, and distribution of the Work otherwise complies with the conditions stated in this License.

5. Submission of Contributions. Unless You explicitly state otherwise, any Contribution intentionally submitted for inclusion in the Work by You to the Licensor shall be under the terms and conditions of this License, without any additional terms or conditions. Notwithstanding the above, nothing herein shall supersede or modify the terms of any separate license agreement you may have executed with Licensor regarding such Contributions.
6. Trademarks. This License does not grant permission to use the trade names, trademarks, service marks, or product names of the Licensor, except as required for reasonable and customary use in describing the origin of the Work and reproducing the content of the NOTICE file.
7. Disclaimer of Warranty. Unless required by applicable law or agreed to in writing, Licensor provides the Work (and each Contributor provides its Contributions) on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied, including, without limitation, any warranties or conditions of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A PARTICULAR PURPOSE. You are solely responsible for determining the appropriateness of using or redistributing the Work and assume any risks associated with Your exercise of permissions under this License.
8. Limitation of Liability. In no event and under no legal theory, whether in tort (including negligence), contract, or otherwise, unless required by applicable law (such as deliberate and grossly negligent acts) or agreed to in writing, shall any Contributor be liable to You for damages, including any direct, indirect, special, incidental, or consequential damages of any character arising as a result of this License or out of the use or inability to use the

Work (including but not limited to damages for loss of goodwill, work stoppage, computer failure or malfunction, or any and all other commercial damages or losses), even if such Contributor has been advised of the possibility of such damages.

9. Accepting Warranty or Additional Liability. While redistributing the Work or Derivative Works thereof, You may choose to offer, and charge a fee for, acceptance of support, warranty, indemnity, or other liability obligations and/or rights consistent with this License. However, in accepting such obligations, You may act only on Your own behalf and on Your sole responsibility, not on behalf of any other Contributor, and only if You agree to indemnify, defend, and hold each Contributor harmless for any liability incurred by, or claims asserted against, such Contributor by reason of your accepting any such warranty or additional liability.

END OF TERMS AND CONDITIONS

APPENDIX: How to apply the Apache License to your work.

To apply the Apache License to your work, attach the following boilerplate notice, with the fields enclosed by brackets "[]" replaced with your own identifying information. (Don't include the brackets!) The text should be enclosed in the appropriate comment syntax for the file format. We also recommend that a file or class name and description of purpose be included on the same "printed page" as the copyright notice for easier identification within third-party archives.

Copyright 2026 Diego Suárez Castro

Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

<http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.